

paper

August 20, 2025

```
[55]: from dynamicalsys import DiscreteDynamicalSystem as dds
      from dynamicalsys import ContinuousDynamicalSystem as cds
      from dynamicalsys import PlotStyler
```

```
[56]: import numpy as np
      import matplotlib.pyplot as plt
      from matplotlib import cm
      import matplotlib.gridspec as gridspec
      import matplotlib as mpl
      from matplotlib.colors import ListedColormap
      import seaborn as sns
      from string import ascii_lowercase
      from joblib import Parallel, delayed
```

1 Basic system definition and simulation

```
[57]: dds.available_models()
```

```
[57]: ['standard map',
      'unbounded standard map',
      'henon map',
      'lozi map',
      'rulkov map',
      'logistic map',
      'standard nontwist map',
      'extended standard nontwist map',
      'leonel map',
      '4d symplectic map']
```

1.1 Standard map

```
[58]: ds = dds(model="standard map")
```

```
[59]: u = [0.05, 0.05] # initial condition
      k = 1.5 # parameter for the standard map
      total_time = 1000000 # total iteration time for each trajectory
```

```
[60]: %%time
trajectory = ds.trajectory(u, total_time, parameters=k)
```

CPU times: user 161 ms, sys: 24.7 ms, total: 186 ms
Wall time: 190 ms

```
[61]: trajectory.shape
```

```
[61]: (1000000, 2)
```

```
[62]: num_ic = 200 # number of initial conditions
np.random.seed(13) # for reproducibility
u = np.random.rand(num_ic, 2) # random initial conditions
k = 1.5 # parameter for the standard map
total_time = 100000 # total iteration time for each trajectory
```

```
[63]: %%time
trajectories = ds.trajectory(u, total_time, parameters=k)
```

CPU times: user 1.33 s, sys: 66.3 ms, total: 1.39 s
Wall time: 434 ms

```
[64]: trajectories.shape
```

```
[64]: (20000000, 2)
```

```
[65]: trajectories = trajectories.reshape(num_ic, total_time, 2)
```

```
[66]: import seaborn as sns
colors = sns.color_palette("hls", num_ic)
np.random.seed(13) # for reproducibility
np.random.shuffle(colors) # shuffle the colors
```

```
[68]: fontsize = 15
ps = PlotStyler(fontsize=fontsize, axes_linewidth=1.1)
ps.apply_style()
fig, ax = plt.subplots(1, 2, sharex=True, sharey=True, figsize=(8, 3.5))
ps.set_tick_padding(ax[0], pad_x=5)
ps.set_tick_padding(ax[1], pad_x=5)

ax[0].plot(trajectory[:, 0], trajectory[:, 1], 'ko', markersize=0.1,
           ↪markeredgewidth=0.0)

for i in range(num_ic):
    ax[1].plot(trajectories[i, :, 0], trajectories[i, :, 1], 'o',
              ↪color=colors[i], markersize=0.2, markedgewidth=0.0)

xbox = 0.0067
```

```

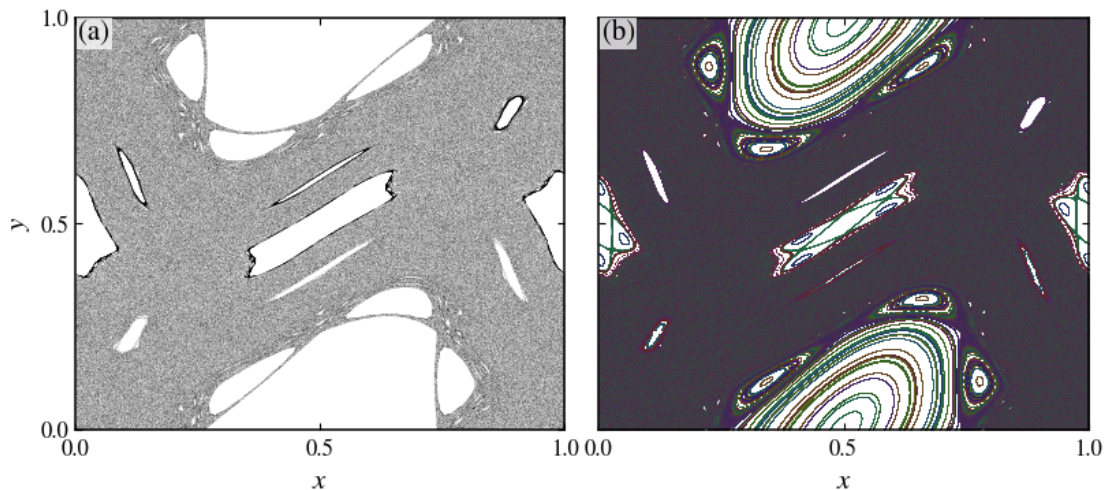
ybox = 0.9457
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

for i in range(2):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

ax[0].set_xticks([0, 0.5, 1])
ax[0].set_yticks([0, 0.5, 1])
ax[0].set_xlim(0, 1)
ax[0].set_ylim(0, 1)
ax[0].set_xlabel(r"$x$")
ax[0].set_ylabel(r"$y$")
ax[1].set_xlabel(r"$x$")
plt.subplots_adjust(left=0.058, bottom=0.12, right=0.985, top=0.98, wspace=0.
    ↪07, hspace=0.2)
plt.savefig("fig1.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



1.2 Hénon map

```
[16]: ds = dds(model="henon map")
```

```
[17]: info = ds.info
info["parameters"]
```

```
[17]: ['a', 'b']
```

```
[18]: u = [0.1, 0.1] # initial condition
a, b = 1.4, 0.3 # parameters for the Henon map
parameters = [a, b]
total_time = 1000000 # total iteration time for each trajectory
transient_time = 500000 # transient time for the Henon map
```

```
[19]: %%time
trajectory = ds.trajectory(u, total_time, parameters=parameters,
    ↪transient_time=transient_time)
```

CPU times: user 237 ms, sys: 23.7 ms, total: 261 ms
Wall time: 283 ms

```
[20]: trajectory.shape
```

```
[20]: (500000, 2)
```

```
[21]: u = [0.1, 0.1] # initial condition
# We are going to change the parameter a and keep b fixed at
b = 0.3
# We define the parameter array with only the b value because a is going to be
    ↪changed
parameters = b
# Define the parameter range
param_range = np.linspace(1, 1.4, 2500)
# Define which parameter will be changed
param_index = 0 # The parameter is the first one (parameters = [a, b])
# Define the total number of iterations (including the transient)
total_time = 8000
# Define the transient iterations
transient_time = 2000
```

```
[23]: %%time
param_values, bifurcation_diagram = ds.bifurcation_diagram(u, param_index,
    ↪param_range, total_time, parameters=parameters,
    ↪transient_time=transient_time)
```

CPU times: user 3.08 s, sys: 36.9 ms, total: 3.12 s
Wall time: 3.14 s

```
[ ]: fontsize = 18
ps = PlotStyler(fontsize=fontsize)
ps.apply_style()
fig, ax = plt.subplots(1, 2, figsize=(10, 3))
ps.set_tick_padding(ax[0], pad_x=5)
ps.set_tick_padding(ax[1], pad_x=5)
ax[0].plot(trajectory[:, 0], trajectory[:, 1], 'ko', markersize=0.2,
    ↪markeredgewidth=0.0)
```

```

ax[0].set_xlim(-1.4, 1.4)
ax[0].set_ylim(-.4, .4)
ax[0].set_xlabel(r"$x$")
ax[0].set_ylabel(r"$y$")

for i in range(bifurcation_diagram.shape[0]):
    ax[1].scatter(param_values[i] * np.ones_like(bifurcation_diagram[i, :]),
        ↪bifurcation_diagram[i, :], c="k", s=0.005, edgecolors="none")

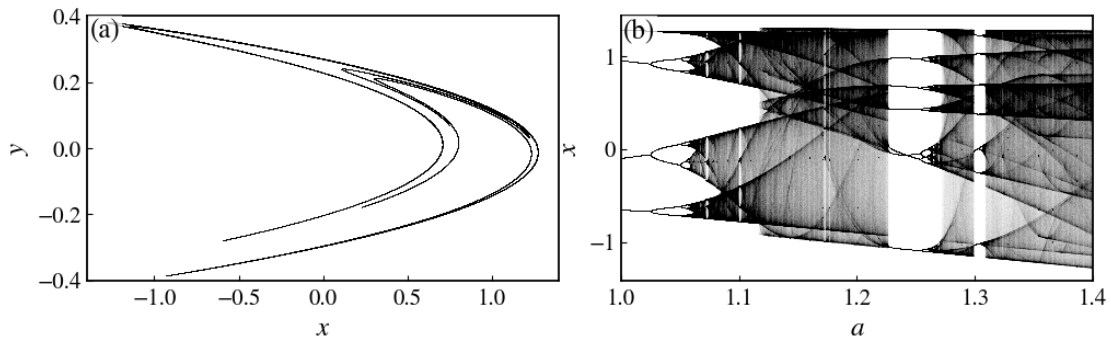
ax[1].set_xlim(param_values.min(), param_values.max())
ax[1].set_xlabel(r"$a$")
ax[1].set_ylabel(r"$x$")

xbox = 0.006
ybox = 0.919
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

for i in range(2):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
        ↪transform=ax[i].transAxes)
plt.subplots_adjust(left=0.068, bottom=0.17, right=0.9875, top=0.975, wspace=0.
    ↪13)
plt.savefig("fig2.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2 Chaotic indicators

2.1 Lyapunov exponents

2.1.1 Final value

```
[24]: from numba import njit
```

```
[25]: @njit
def dakrm(u, parameters):
    k, a, gamma = parameters
    x, y = u

    y_new = (1 - gamma) * y + k * (np.sin(x) + a * np.sin(2 * x + np.pi / 2))
    x_new = (x + y_new) % (2 * np.pi)
    return np.array([x_new, y_new])

@njit
def dakrm_jacobian(u, parameters, *args):
    k, a, gamma = parameters
    x, y = u

    dFdx = k * (np.cos(x) + 2 * a * np.cos(2 * x + np.pi / 2))
    dFdy = 1 - gamma

    return np.array([
        [1 + dFdx, dFdy],
        [dFdx, dFdy]
    ])
```

```
[33]: ds = dds(mapping=dakrm, jacobian=dakrm_jacobian, system_dimension=2,
↳number_of_parameters=3)
```

```
[34]: # Initial condition
u = np.array([1.78, 0.0])
# Parameters
gamma = 0.8
grid_size = 1000
k = np.linspace(0, 30, grid_size)
a = np.linspace(0, 1, grid_size)
K, A = np.meshgrid(k, a)
# Total number of iterations (including the transient)
total_time = 10000
# Transient iterations
transient_time = 5000
```

```
[35]: k = 8
a = 0.47
gamma = 0.8
parameters = [k, a, gamma]
```

```
[37]: %%time
ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
CPU times: user 3.92 ms, sys: 58 s, total: 3.97 ms
Wall time: 4.15 ms
```

```
[37]: array([-0.35202562, -1.25741229])
```

```
[38]: ds.period(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
[38]: 2
```

```
[39]: k = 8
      a = 0.6
      gamma = 0.8
      parameters = np.array([k, a, gamma])
```

```
[40]: %%time
      ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
CPU times: user 2.16 ms, sys: 36 s, total: 2.19 ms
Wall time: 2.28 ms
```

```
[40]: array([ 1.57224186, -3.18167977])
```

```
[41]: ds.period(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
[41]: -1
```

```
[42]: ds = dds(mapping=dakrm, system_dimension=2, number_of_parameters=3)
```

```
[43]: # Initial condition
      u = np.array([1.78, 0.0])
      # Parameters
      k = 8
      a = 0.6
      gamma = 0.8
      parameters = np.array([k, a, gamma])
      # Total number of iterations (including the transient)
      total_time = 10000
      # Transient iterations
      transient_time = 5000
```

```
[44]: %%time
      ds.lyapunov(u, total_time, parameters=parameters, transient_time=transient_time)
```

```
CPU times: user 413 ms, sys: 8.68 ms, total: 422 ms
Wall time: 444 ms
```

```
[44]: array([ 1.5740678 , -3.18114158])
```

```
[45]: %%time
      lyapunov = np.array(Parallel(n_jobs=-1)(delayed(ds.lyapunov)(u, total_time,
      ↪ parameters=np.array([K[i, j], A[i, j], gamma]), transient_time=1000) for i
      ↪ in range(grid_size) for j in range(grid_size)))
```

CPU times: user 2min 24s, sys: 7.93 s, total: 2min 32s
Wall time: 25min 31s

```
[46]: lyapunov.shape
```

```
[46]: (1000000, 2)
```

```
[50]: %%time
period = np.array(Parallel(n_jobs=-1)(delayed(ds.period)(u, total_time,
    ↪parameters=np.array([k_val, a_val, gamma]), transient_time=transient_time)
    ↪for k_val, a_val in zip(K.flatten(), A.flatten()))))
```

CPU times: user 42 s, sys: 1.59 s, total: 43.6 s
Wall time: 7min 56s

```
[54]: import seaborn as sns
cmap = sns.diverging_palette(145, 300, s=60, as_cmap=True)
```

```
[55]: ps = PlotStyler(fontsize=18, ticks_on_all_sides=False)
ps.apply_style()
fig, ax = plt.subplots(1, 2, figsize=(10, 4), sharex=True, sharey=True)
ps.set_tick_padding(ax[0], pad_x=5)
ps.set_tick_padding(ax[1], pad_x=5)

hm = ax[0].pcolor(A, K, lyapunov[:, 0].reshape((grid_size, grid_size)),
    ↪cmap=cmap, vmin=-1, vmax=1)
plt.colorbar(hm, aspect=40, pad=0.02, label=r'$\lambda_1$', location="top")
ax[0].set_xlabel(r'$a$')
ax[0].set_ylabel(r'$k$')

aux_period = np.asarray(period, dtype=np.float64).reshape(grid_size, grid_size)
aux_period[np.where(aux_period == -1)] = np.nan
cmaplist = sns.color_palette("tab10", 10)
cmap = mpl.colors.ListedColormap(cmaplist)
bounds = np.linspace(0.5, 10.5, 11)
norm = mpl.colors.BoundaryNorm(bounds, cmap.N)
hm = ax[1].pcolormesh(A, K, aux_period, cmap=cmap, norm=norm)
ticks = np.arange(1, 11, 1)
cbar = plt.colorbar(hm, ticks=ticks, label="Period", aspect=40, pad=0.02,
    ↪location="top")
ticks = list(ticks)
ticks[-1] = "$\geq 10$"
cbar.ax.set_xticklabels(ticks)
cbar.ax.minorticks_off()
ax[1].set_xlabel(r'$a$')

xbox = 0.0058
```

```

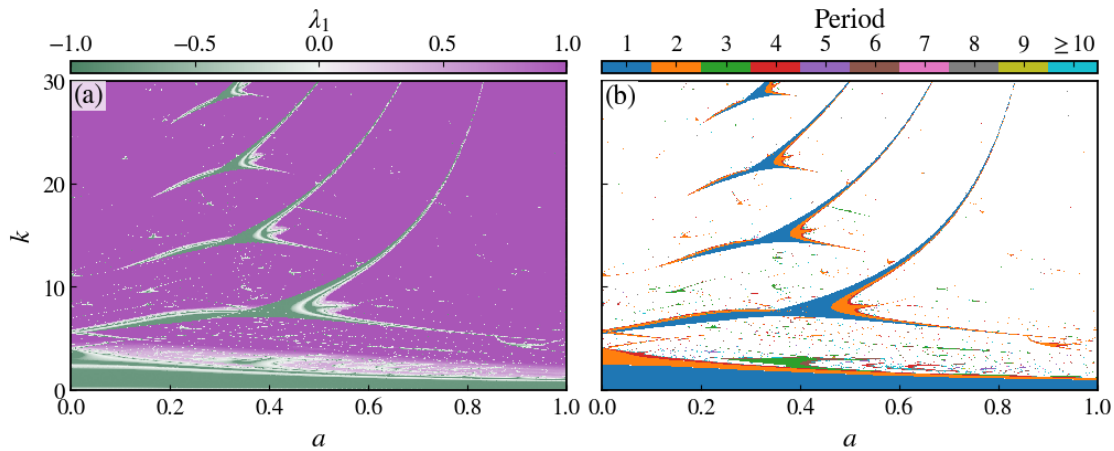
ybox = 0.93
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

for i in range(2):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

plt.subplots_adjust(left=0.055, bottom=0.125, right=0.987, top=0.9675, wspace=0.
    ↪07)
plt.savefig("fig3.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2.1.2 Whole history

```
[3]: ds = dds(model="standard map")
```

```

[4]: k = 0.9
total_time = 1000000
u = np.array([[0.26, 0.0],
              [0.4, 0.],
              [0, 0.45],
              [0.1, 0.25],
              [0.1, 0.68],
              [0.06, 0.05],
              [0, 0.3],
              [0, 0.6],
              [0, 0.7]])
num_ic = u.shape[0] # number of initial conditions

```

```
[7]: %%time
ts = ds.trajectory(u, total_time, parameters=k)
```

CPU times: user 932 ms, sys: 81.2 ms, total: 1.01 s
Wall time: 682 ms

```
[8]: ts = ts.reshape(num_ic, total_time, 2)
```

```
[20]: total_time = 100000000
sample_times = np.unique(np.logspace(np.log10(1), np.log10(total_time), 1000).
    ↪astype(int))
```

```
[21]: %%time
lyapunovs = np.array(Parallel(n_jobs=-1)(delayed(ds.lyapunov)(u[i], total_time,
    ↪parameters=k, return_history=True, sample_times=sample_times) for i in
    ↪range(num_ic)))
```

CPU times: user 147 ms, sys: 50.4 ms, total: 197 ms
Wall time: 2min 29s

```
[22]: lyapunovs.shape
```

```
[22]: (9, 836, 2)
```

```
[23]: ps = PlotStyler(fontsize=18, ticks_on_all_sides=False, markersize=0.2,
    ↪markeredgewidth=0)
ps.apply_style()
# Create figure
fig = plt.figure(figsize=(10, 3))
colors = sns.color_palette("tab10", num_ic)
# Create GridSpec with 1 row and 3 columns
gs = gridspec.GridSpec(1, 3)
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1:])
ax = np.array([ax1, ax2], dtype=object)
[ps.set_tick_padding(ax[i], pad_x=6) for i in range(ax.shape[0])]

for i in range(num_ic):
    ax[0].plot(ts[i, :, 0], ts[i, :, 1], 'o', color=colors[i])
    ax[1].plot(sample_times, lyapunovs[i, :, 0], '-', color=colors[i])

ax[0].set_xlim(0, 1)
ax[0].set_ylim(0, 1)
ax[0].set_xlabel(r"$x$")
ax[0].set_ylabel(r"$y$")
ax[0].set_xticks([0, 0.5, 1])
ax[0].set_yticks([0, 0.5, 1])
```

```

ax[1].set_xlabel(r"$n$")
ax[1].set_ylabel(r"$\lambda_1$")
ax[1].set_xlim(sample_times[0], sample_times[-1])
ax[1].set_yscale("log")
ax[1].set_xscale("log")

xbox = [0.01003, 0.004]
ybox = [0.919, 0.919]
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

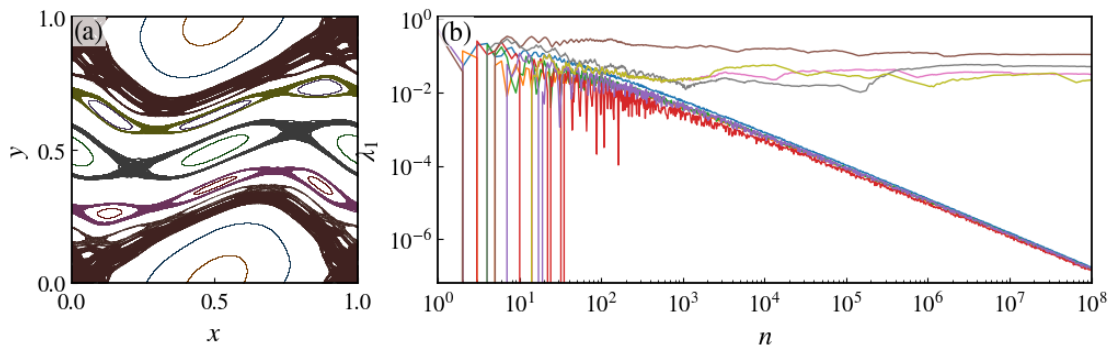
for i in range(2):
    ax[i].text(xbox[i], ybox[i], f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

plt.subplots_adjust(left=0.054, bottom=0.165, right=0.985, top=0.97, wspace=0.
    ↪28, hspace=0.2)

plt.savefig("fig4.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2.1.3 Finite-time Lyapunov exponent

```
[45]: ds = dds(model="standard map")
```

```
[46]: u = [0.5, 0.25]
nk = 5000
k = np.linspace(0, 5, nk)
total_time = 5000
```

```
[ ]: %%time
lypnvs_vs_k = np.array([ds.lyapunov(u, total_time, parameters=k[i]) for i in
    ↪range(nk)])
```

CPU times: user 9.49 s, sys: 56 ms, total: 9.55 s
Wall time: 9.56 s

```
[48]: u = [0.05, 0.05]
      parameter = 1.5
      # The total number of iterations for the FTRTE computation is
      total_time = 100000000
      # and the size of the windows is
      finite_time = 200
```

```
[49]: %%time
      ftle = ds.finite_time_lyapunov(u, total_time, finite_time, parameters=parameter)
```

CPU times: user 38.8 s, sys: 204 ms, total: 39 s
Wall time: 39 s

```
[50]: total_time = 2000000
      # and the size of the windows is
      finite_time = 200
```

```
[51]: %%time
      _, points = ds.finite_time_lyapunov(u, total_time, finite_time,
      ↪ parameters=parameter, return_points=True)
```

CPU times: user 839 ms, sys: 36.3 ms, total: 875 ms
Wall time: 846 ms

```
[52]: ts = ds.trajectory(points, finite_time, parameters=parameter)
      ts = ts.reshape(points.shape[0], finite_time, 2)
```

```
[55]: fontsize = 18
      ps = PlotStyler(fontsize=18, ticks_on_all_sides=False)
      ps.apply_style()
      fig, ax = plt.subplots(1, 3, figsize=(10, 3.5))
      [ps.set_tick_padding(ax[i], pad_x=6) for i in range(ax.shape[0])]

      ax[0].plot(k, lypnvs_vs_k[:, 0], "k")
      ax[0].set_xlim(k.min(), k.max())
      ax[0].set_xticks([0, 1, 2, 3, 4, 5])
      ax[0].set_xlabel("$k$")
      ax[0].set_ylabel(r"$\lambda_1$")
      ax[0].set_ylim(-0.01, 1.02)

      for i in range(points.shape[0]):
          ax[1].scatter(ts[i, :, 0], ts[i, :, 1], c=ftle[i, 0] * np.
          ↪ ones(finite_time), cmap="nipy_spectral", s=0.05, edgecolors="none", vmin=0,
          ↪ vmax=ftle[:, 0].max())
```

```

ax[1].set_xlim(0, 1)
ax[1].set_ylim(0, 1)
ax[1].set_xlabel(r"$x$")
ax[1].set_ylabel(r"$y$")
ax[1].set_xticks([0, 0.5, 1])
ax[1].set_yticks([0, 0.5, 1])

counts, bins, patches = ax[2].hist(ftle[:, 0], bins="auto", edgecolor='none',
    ↪density=True)
# Compute bin centers
bin_centers = 0.5 * (bins[:-1] + bins[1:])

# Normalize bin centers for colormap
norm = plt.Normalize(0, bin_centers.max())
colormap = cm.nipy_spectral # You can choose any colormap you like

# Apply color based on bin center (x position)
for center, patch in zip(bin_centers, patches):
    color = colormap(norm(center))
    patch.set_facecolor(color)

ax[2].set_xlim(bins[0], 0.5)
ax[2].set_xlabel(r"$\lambda_1(200)$")
ax[2].set_ylabel("Density")

ax_ins = ax[2].inset_axes([0.05, 0.5, 0.475, 0.4]) # [left, bottom, width,
    ↪height]

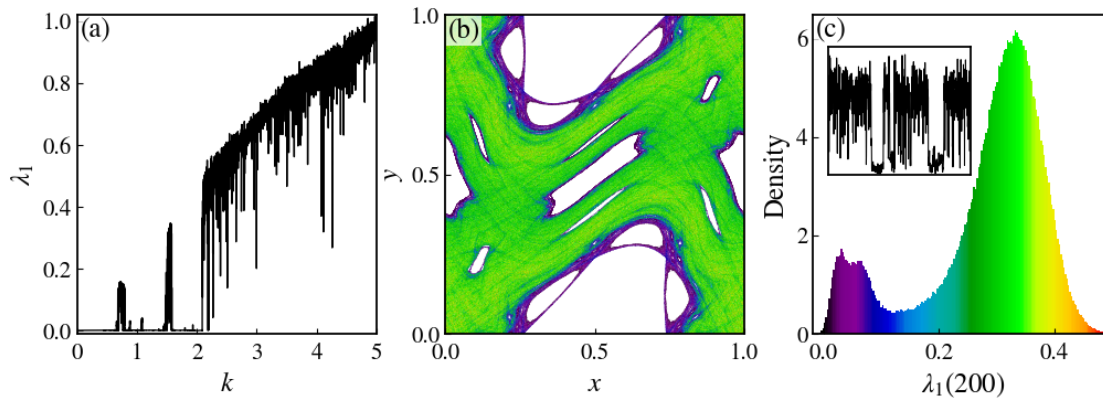
ii = np.arange(ftle.shape[0]) / 1e3
ax_ins.plot(ii, ftle[:, 0], "k", lw=0.75)
ax_ins.set_xlim(1, 2)
ax_ins.set_ylim(0, 0.5)
ax_ins.set_xticks([])
ax_ins.set_yticks([])

xbox = 0.0095
ybox = 0.9318
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}
for i in range(3):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)
plt.subplots_adjust(left=0.06, bottom=0.15, right=0.9975, top=0.975, wspace=0.
    ↪23, hspace=0.2)

plt.savefig("fig5.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2.2 Linear dependence index

```
[7]: ds = dds(model="4D symplectic map")
```

```
[8]: info = ds.info()
info["parameters"]
```

```
[8]: ['eps1', 'eps2', 'xi']
```

```
[9]: u = np.array([[0.5, 0.0, 0.5, 0.0],
                  [3.0, 0.0, 0.5, 0.0]])
parameters = np.array([0.5, 0.1, 0.001], dtype=np.float64)
total_time = int(1e6)
sample_times = np.unique(np.logspace(np.log10(1), np.log10(total_time), 5000).
    ↪ astype(int))
```

```
[ ]: %%time
lyapunovs = np.array(Parallel(n_jobs=-1)(delayed(ds.lyapunov)(u[i], total_time,
    ↪ parameters=parameters, return_history=True, sample_times=sample_times) for i
    ↪ in range(u.shape[0])))
```

CPU times: user 34.9 ms, sys: 109 ms, total: 144 ms

Wall time: 9.58 s

```
[11]: lyapunovs.shape
```

```
[11]: (2, 3229, 4)
```

```
[12]: k = [2, 3, 4]
total_time = int(1e5)
times = np.arange(1, total_time + 1)
ldi = np.zeros((u.shape[0], total_time, len(k)))
```

```
[ ]: %%time
for i in range(len(k)):
    for j in range(u.shape[0]):
        ldi[j, :, i] = ds.LDI(u[j], total_time, k[i], parameters=parameters,
        ↪return_history=True)
```

```
/opt/anaconda3/lib/python3.12/site-packages/numba/core/utils.py:661:
NumbaExperimentalFeatureWarning: First-class function type feature is
experimental
    warnings.warn("First-class function type feature is experimental",
/opt/anaconda3/lib/python3.12/site-packages/numba/core/utils.py:661:
NumbaExperimentalFeatureWarning: First-class function type feature is
experimental
    warnings.warn("First-class function type feature is experimental",
CPU times: user 9.77 s, sys: 90.4 ms, total: 9.86 s
Wall time: 9.91 s
```

```
[14]: ldi.shape
```

```
[14]: (2, 100000, 3)
```

```
[ ]: ps = PlotStyler(fontsize=20, ticks_on_all_sides=False)
ps.apply_style()
fig, ax = plt.subplots(1, 2, figsize=(12, 3.5))
ps.set_tick_padding(ax[0], pad_x=5)
ps.set_tick_padding(ax[1], pad_x=5)

ax[0].plot(sample_times, lyapunovs[0, :, 0], '-', color="red", lw=0.9,
    ↪label=r"$\lambda_1^{(1)}$")
ax[0].plot(sample_times, lyapunovs[1, :, 0], '-', color="blue", lw=0.9,
    ↪label=r"$\lambda_1^{(2)}$")

colors = [['r', 'r', 'fuchsia', 'darkviolet'],
          ['b', 'b', 'royalblue', 'navy']]
for i in range(u.shape[0]):
    ax[1].plot(times, ldi[i, :, 0], "-", color=colors[i][1], lw=0.9,
    ↪label=f"LDI$_2^{\{(i + 1)\}}$")
    ax[1].plot(times, ldi[i, :, 1], "-", color=colors[i][2], lw=0.9,
    ↪label=f"LDI$_3^{\{(i + 1)\}}$")
    ax[1].plot(times, ldi[i, :, 2], "-", color=colors[i][3], lw=0.9,
    ↪label=f"LDI$_4^{\{(i + 1)\}}$")

ax[0].legend(loc="upper right", frameon=False)
ax[1].legend(loc="lower left", frameon=False)

ax[0].set_xlim(sample_times[0], sample_times[-1])
```

```

ax[0].set_xlabel(r"$n$")
ax[0].set_ylabel(r"$\lambda_1$")
ax[0].set_xscale("log")
ax[0].set_yscale("log")

ax[1].set_xlim(times[0], times[-1])
ax[1].set_ylim(1e-16, np.sqrt(2))
ax[1].set_xlabel(r"$n$")
ax[1].set_ylabel(r"$LDI_k$")
ax[1].set_xscale("log")
ax[1].set_yscale("log")

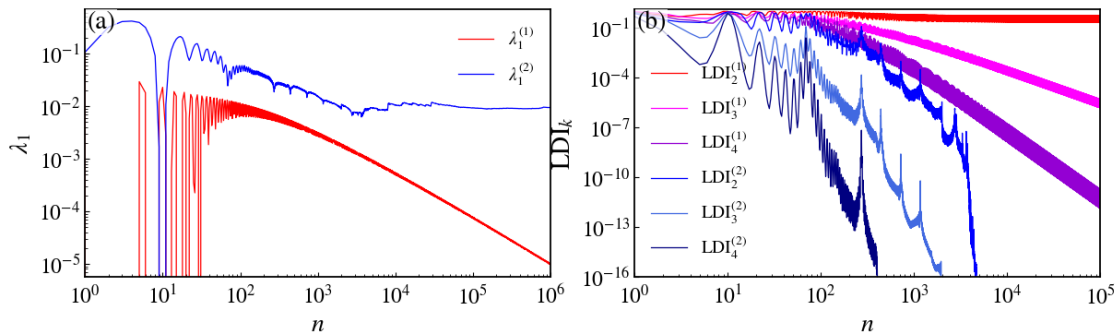
xbox = 0.0049
ybox = 0.9265
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

for i in range(2):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

plt.subplots_adjust(left=0.0675, bottom=0.16, right=0.985, top=0.99, wspace=0.
    ↪18, hspace=0.2)
plt.savefig("fig5.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



```

[16]: total_time = 1000
      nruns = 500
      from time import time

```

```

[ ]: exe_times = []
     for _ in range(nruns):
         time_ini = time()
         ds.LDI(u[0], total_time, 2, parameters=parameters)

```

```

time_end = time()
exe_times.append(time_end - time_ini)
LDI_time = np.mean(exe_times)
print(f"Execution time for LDI: {np.mean(exe_times):.5f} +- {np.std(exe_times):.5f} seconds")

```

Execution time for LDI: 0.02749 +- 0.00064 seconds

```

[ ]: exe_times = []
for _ in range(nruns):
    time_ini = time()
    ds.SALI(u[0], total_time, parameters=parameters)
    time_end = time()
    exe_times.append(time_end - time_ini)
SALI_time = np.mean(exe_times)
print(f"Execution time for SALI: {np.mean(exe_times):.6f} +- {np.std(exe_times):.6f} seconds")

```

Execution time for SALI: 0.002777 +- 0.041096 seconds

```

[19]: LDI_time, SALI_time, LDI_time / SALI_time

```

```

[19]: (0.027492702960968018, 0.002777045249938965, 9.899983790891511)

```

2.3 Weighted Birkhoff averagas

```

[3]: ds = dds(model="standard map")

```

```

[ ]: k = 1.5
y = np.linspace(0, 1, 20001)
x = 0.5 * np.ones_like(y)
u = np.array([x, y]).T
total_time = 10000

```

```

[ ]: %%time
dig = np.array([ds.dig(u[i], total_time, parameters=k) for i in range(u.shape[0])])

```

/Users/mrolims/Library/CloudStorage/Dropbox/Física/Pesquisa/pycandy/src/pycandy/dynamical_indicators.py:528: RuntimeWarning: divide by zero encountered in log10

```

    return - np.log10(abs(WB0 - WB1))

```

CPU times: user 8.47 s, sys: 179 ms, total: 8.65 s

Wall time: 8.66 s

```

[ ]: %%time
dig2 = np.array([ds.dig(u[i], total_time, parameters=k, func=lambda x: np.sin(2*np.pi * x[:, 0])) for i in range(u.shape[0])])

```

CPU times: user 8.56 s, sys: 180 ms, total: 8.74 s
Wall time: 8.76 s

```
[ ]: %%time
dig3 = np.array([ds.dig(u[i], total_time, parameters=k, func=lambda x: np.sin(2 *
↳ np.pi * (x[:, 0] + x[:, 1]))) for i in range(u.shape[0])])
```

CPU times: user 8.64 s, sys: 138 ms, total: 8.78 s
Wall time: 8.79 s

```
[231]: grid_size = 1000
x = np.linspace(0, 1, grid_size)
y = np.linspace(0, 1, grid_size)
X, Y = np.meshgrid(x, y)
u = np.array([X.flatten(), Y.flatten()]).T
k = 1.5
N = 10000
```

```
[ ]: %%time
dig = np.array(Parallel(n_jobs=-1)(delayed(ds.dig)(u[i], total_time,
↳ parameters=k) for i in range(u.shape[0])))
```

```
/Users/mrolims/Library/CloudStorage/Dropbox/Física/Pesquisa/pycandy/src/pycandy
/dynamical_indicators.py:528: RuntimeWarning: divide by zero encountered in
log10
    return - np.log10(abs(WB0 - WB1))
/Users/mrolims/Library/CloudStorage/Dropbox/Física/Pesquisa/pycandy/src/pycandy
/dynamical_indicators.py:528: RuntimeWarning: divide by zero encountered in
log10
    return - np.log10(abs(WB0 - WB1))
/Users/mrolims/Library/CloudStorage/Dropbox/Física/Pesquisa/pycandy/src/pycandy
/dynamical_indicators.py:528: RuntimeWarning: divide by zero encountered in
log10
    return - np.log10(abs(WB0 - WB1))
/Users/mrolims/Library/CloudStorage/Dropbox/Física/Pesquisa/pycandy/src/pycandy
/dynamical_indicators.py:528: RuntimeWarning: divide by zero encountered in
log10
    return - np.log10(abs(WB0 - WB1))

CPU times: user 28.2 s, sys: 877 ms, total: 29 s
Wall time: 1min 36s
```

```
[233]: dig = dig.reshape(grid_size, grid_size)
```

```
[ ]: %%time
dig2 = np.array(Parallel(n_jobs=-1)(delayed(ds.dig)(u[i], total_time,
↳ parameters=k, func=lambda x: np.sin(2 * np.pi * x[:, 0])) for i in range(u.
↳ shape[0])))
```

CPU times: user 4min 37s, sys: 2.84 s, total: 4min 40s
Wall time: 4min 40s

```
[180]: dig2 = dig2.reshape(grid_size, grid_size)
```

```
[181]: dig2.shape
```

```
[181]: (1000, 1000)
```

```
[ ]: %%time
dig3 = np.array(Parallel(n_jobs=-1)(delayed(ds.dig)(u[i], total_time,
↳ parameters=k, func=lambda x: np.sin(2 * np.pi * (x[:, 0] + x[:, 1]))) for i in
↳ range(u.shape[0])))
```

CPU times: user 5min 13s, sys: 3.75 s, total: 5min 17s
Wall time: 5min 18s

```
[183]: dig3 = dig3.reshape(grid_size, grid_size)
```

```
[234]: # Remove inf from dig: substitute with the mean of the neighbors
def remove_inf(dig):
    dig[np.isinf(dig)] = np.nan
    for i in range(dig.shape[0]):
        for j in range(dig.shape[1]):
            if np.isnan(dig[i, j]):
                neighbors = []
                if i > 0:
                    neighbors.append(dig[i - 1, j])
                if i < dig.shape[0] - 1:
                    neighbors.append(dig[i + 1, j])
                if j > 0:
                    neighbors.append(dig[i, j - 1])
                if j < dig.shape[1] - 1:
                    neighbors.append(dig[i, j + 1])
                dig[i, j] = np.nanmean(neighbors)
    return dig
dig_ri = remove_inf(dig)
dig2_ri = remove_inf(dig2)
dig3_ri = remove_inf(dig3)
```

```
[ ]: ps = PlotStyler(fontsize=18, ticks_on_all_sides=False)
ps.apply_style()

fig, ax = plt.subplots(1, 3, figsize=(10, 4), sharex=True, sharey=True)
ps.set_tick_padding(ax[0], pad_x=6)
ps.set_tick_padding(ax[1], pad_x=6)
ps.set_tick_padding(ax[2], pad_x=6)
```

```

cmap = "nipy_spectral"
# cmap = sns.color_palette("magma", as_cmap=True)
hm = ax[0].pcolor(X, Y, dig_ri, cmap=cmap, vmin=0, vmax=dig_ri.max())
cbar = plt.colorbar(hm, aspect=30, pad=0.02, label="dig", location="top")
cbar.ax.set_xticks([0, 4, 8, 12, 16])
ax[0].set_xlabel(r"$x$")
ax[0].set_ylabel(r"$y$")

hm = ax[1].pcolor(X, Y, dig2_ri, cmap=cmap, vmin=0, vmax=dig2_ri.max())
cbar = plt.colorbar(hm, aspect=30, pad=0.02, label="dig", location="top")
cbar.ax.set_xticks([0, 5, 10, 15, 20])
ax[1].set_xlabel(r"$x$")
hm = ax[2].pcolor(X, Y, dig3_ri, cmap=cmap, vmin=0, vmax=dig3_ri.max())
cbar = plt.colorbar(hm, aspect=30, pad=0.02, label="dig", location="top")
cbar.ax.set_xticks([0, 5, 10, 15, 20])
ax[2].set_xlabel(r"$x$")

xbox = 0.009
ybox = 0.9298
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

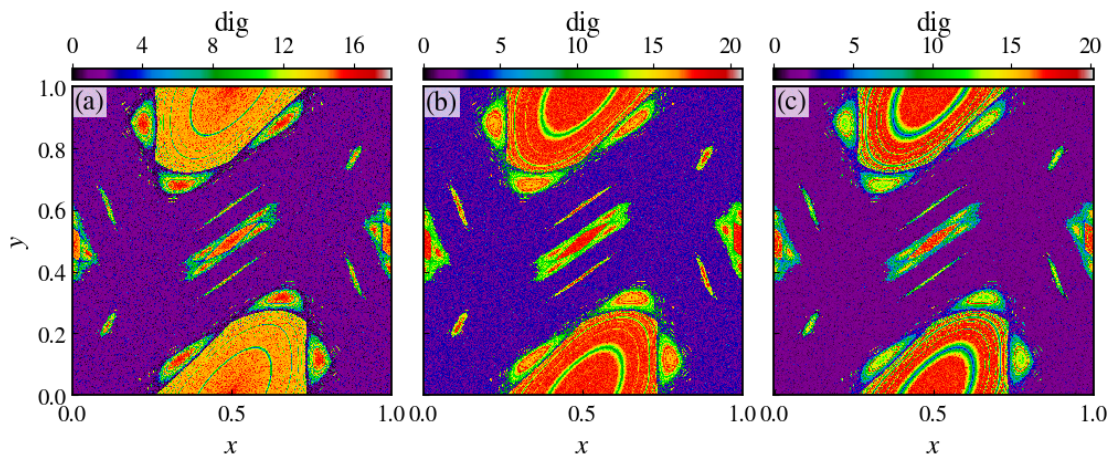
for i in range(3):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

plt.subplots_adjust(left=0.06, bottom=0.13, right=0.985, top=0.97, wspace=0.1,
    ↪hspace=0.2)

plt.savefig("fig6.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2.4 Recurrence time entropy

```
[77]: ds = dds(model="standard map")
```

2.4.1 Recurrence matrix

```
[98]: u = [[0.05, 0.05],  
          [0.35, 0.0],  
          [0.42, 0.2]]  
k = 1.5  
N = 1000
```

```
[99]: %%time  
recmats = [ds.recurrence_matrix(u[i], N, parameters=k) for i in range(len(u))]
```

CPU times: user 4.44 ms, sys: 1.48 ms, total: 5.93 ms

Wall time: 5.12 ms

```
[100]: total_time = 250000
```

```
[101]: %%time  
ts = ds.trajectory(u, total_time, parameters=k)
```

CPU times: user 59.7 ms, sys: 6.12 ms, total: 65.8 ms

Wall time: 22.9 ms

```
[102]: ts.shape
```

```
[102]: (750000, 2)
```

```
[103]: ts = ts.reshape(len(u), total_time, 2)
```

```
[104]: ps = PlotStyler(fontsize=21, ticks_on_all_sides=False)  
ps.apply_style()  
  
fig, ax = plt.subplots(1, 4, figsize=(12, 3.5))  
ps.set_tick_padding(ax[0], pad_x=5)  
ps.set_tick_padding(ax[1], pad_x=5)  
ps.set_tick_padding(ax[2], pad_x=5)  
ps.set_tick_padding(ax[3], pad_x=5)  
  
colors = sns.color_palette("hls", len(u))  
  
for i in range(len(u)):  
    # ax[0].scatter(u[i][0], u[i][1], c=colors[i], s=10, edgecolors="none")  
    ax[0].plot(ts[i, :, 0], ts[i, :, 1], 'o', markersize=0.3, markeredgewidth=0.  
↪0, color=colors[i])
```

```

x = np.where(recmats[i] == 1)[0]
y = np.where(recmats[i] == 1)[1]

ax[1 + i].scatter(x, y, s=0.5, color=colors[i], edgecolors="none")
ax[1 + i].set_xlim(0, N)
ax[1 + i].set_ylim(0, N)
ax[1 + i].set_xlabel(r"$i$")
ax[i + 1].set_yticks([0, 500, 1000])
ax[i + 1].set_xticks([0, 500, 1000])
ax[0].set_xlim(0, 1)
ax[0].set_ylim(0, 1)
ax[0].set_xlabel(r"$x$")
ax[0].set_ylabel(r"$y$")
ax[0].set_xticks([0, 0.5, 1])
ax[0].set_yticks([0, 0.5, 1])
label = ax[1].set_ylabel(r"$j$", rotation=0, labelpad=0.1)
# label.set_position((1, 0.6)) # (x, y) in axis coordinates
# ax[2].set_yticklabels([])
# ax[3].set_yticklabels([])

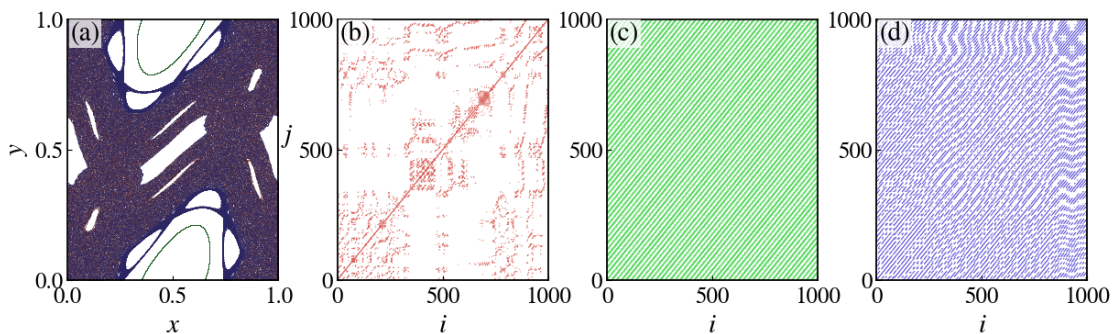
xbox = 0.012
ybox = 0.921
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}
for i in range(4):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)

plt.subplots_adjust(left=0.0525, bottom=0.161, right=0.98, top=0.972, wspace=0.
    ↪28, hspace=0.2)

plt.savefig("fig8.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



```
[6]: u = [0.5, 0.25]
nk = 5000
k = np.linspace(0, 5, nk)
total_time = 5000
```

```
[7]: %%time
rte = Parallel(n_jobs=-1)(delayed(ds.recurrence_time_entropy)(u, total_time,
↳ parameters=k[i]) for i in range(k.shape[0]))
rte = np.array(rte)
```

CPU times: user 2.8 s, sys: 148 ms, total: 2.95 s
Wall time: 19.2 s

```
[9]: u = [0.05, 0.05]
parameter = 1.5
# The total number of iterations for the FTRTE computation is
total_time = 100000000
# and the size of the windows is
finite_time = 200
```

```
[11]: %%time
ftrte = ds.finite_time_recurrence_time_entropy(u, total_time, finite_time,
↳ parameters=parameter)
```

CPU times: user 34.8 s, sys: 264 ms, total: 35.1 s
Wall time: 35.2 s

```
[39]: total_time = 2000000
# and the size of the windows is
finite_time = 200
```

```
[40]: %%time
_, points = ds.finite_time_recurrence_time_entropy(u, total_time, finite_time,
↳ parameters=parameter, return_points=True)
```

CPU times: user 796 ms, sys: 77.6 ms, total: 874 ms
Wall time: 804 ms

```
[41]: ts = ds.trajectory(points, finite_time, parameters=parameter)
ts = ts.reshape(len(points), finite_time, 2)
```

```
[44]: fontsize = 18
ps = PlotStyler(fontsize=fontsize, ticks_on_all_sides=False)
ps.apply_style()
fig, ax = plt.subplots(1, 3, figsize=(10, 3.5))
[ps.set_tick_padding(ax[i], pad_x=6) for i in range(ax.shape[0])]

ax[0].plot(k, rte, "k", lw=0.5)
ax[0].set_xlim(k.min(), k.max())
```

```

ax[0].set_xticks([0, 1, 2, 3, 4, 5])
ax[0].set_ylim(0, 7)
ax[0].set_xlabel(r"$k$")
ax[0].set_ylabel(r"RTE")

for i in range(points.shape[0]):
    ax[1].scatter(ts[i, :, 0], ts[i, :, 1], c=ftrte[i] * np.ones(finite_time),
        cmap="nipy_spectral", s=0.05, edgecolors="none", vmin=0, vmax=ftrte.max())

ax[1].set_xlim(0, 1)
ax[1].set_ylim(0, 1)
ax[1].set_xlabel(r"$x$")
ax[1].set_ylabel(r"$y$")
ax[1].set_xticks([0, 0.5, 1])
ax[1].set_yticks([0, 0.5, 1])

counts, bins, patches = ax[2].hist(ftrte, bins="auto", edgecolor='none',
    density=True)
# Compute bin centers
bin_centers = 0.5 * (bins[:-1] + bins[1:])

# Normalize bin centers for colormap
norm = plt.Normalize(0, bin_centers.max())
colormap = cm.nipy_spectral # You can choose any colormap you like

# Apply color based on bin center (x position)
for center, patch in zip(bin_centers, patches):
    color = colormap(norm(center))
    patch.set_facecolor(color)

ax[2].set_xlim(bins[0], bins[-1])
ax[2].set_xlabel(r"RTE(200)")
ax[2].set_ylabel("Density")
ax[2].set_xticks([0, 1, 2, 3, 4, 5])
ax[2].set_yticks([0, 0.5, 1, 1.5])

ax_ins = ax[2].inset_axes([0.05, 0.35, 0.65, 0.525]) # [left, bottom, width,
    height]
ii = np.arange(ftrte.shape[0]) / 1e3
ax_ins.plot(ii, ftrte, "k", lw=0.75)
ax_ins.set_xlim(1, 2)
ax_ins.set_xticks([])
ax_ins.set_yticks([])

xbox = 0.0095
ybox = 0.9318
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}

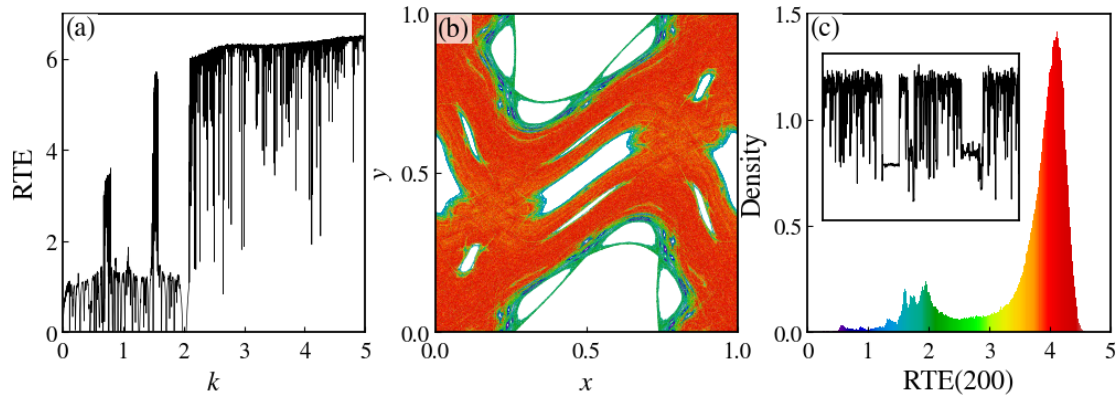
```

```

for i in range(3):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
    ↪transform=ax[i].transAxes)
plt.subplots_adjust(left=0.045, bottom=0.15, right=0.995, top=0.975, wspace=0.
    ↪23, hspace=0.2)
plt.savefig("fig9.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



2.5 Hurst exponent

```

[105]: ds = dds(model="standard map")

```

```

[106]: u = [0.5, 0.25]
nk = 5000
k = np.linspace(0, 5, nk)
total_time = 5000

```

```

[61]: %%time
HE = Parallel(n_jobs=-1)(delayed(ds.hurst_exponent)(u, total_time,
    ↪parameters=k[i]) for i in range(k.shape[0]))
HE = np.array(HE)

```

CPU times: user 5.19 s, sys: 227 ms, total: 5.42 s

Wall time: 1min 9s

```

[113]: u = [0.05, 0.05]
parameter = 1.5
# The total number of iterations for the FTRTE computation is
total_time = 100000000
# and the size of the windows is
finite_time = 200

```

```
[114]: %time
ftHE = ds.finite_time_hurst_exponent(u, total_time, finite_time,
    ↪parameters=parameter)
```

CPU times: user 2 s, sys: 1 s, total: 3 s
Wall time: 4.77 s

```
[75]: ftHE_avg = (ftHE[:, 0] + ftHE[:, 1]) / 2
```

```
[91]: total_time = 2000000
# and the size of the windows is
finite_time = 200
```

```
[92]: %%time
_, points = ds.finite_time_hurst_exponent(u, total_time, finite_time,
    ↪parameters=parameter, return_points=True)
```

CPU times: user 2.72 s, sys: 76 ms, total: 2.79 s
Wall time: 2.78 s

```
[93]: fontsize = 18
ps = PlotStyler(fontsize=fontsize, ticks_on_all_sides=False)
ps.apply_style()
fig, ax = plt.subplots(1, 3, figsize=(10, 3.5))
[ps.set_tick_padding(ax[i], pad_x=6) for i in range(ax.shape[0])]

ax[0].plot(k, (HE[:, 0] + HE[:, 1]) / 2, "k", lw=0.5)
ax[0].set_xlim(k.min(), k.max())
ax[0].set_xticks([0, 1, 2, 3, 4, 5])
ax[0].set_ylim(0, 1)
ax[0].set_xlabel(r"$k$")
ax[0].set_ylabel(r"$H$")

for i in range(points.shape[0]):
    ax[1].scatter(ts[i, :, 0], ts[i, :, 1], c=ftHE_avg[i] * np.
    ↪ones(finite_time), cmap="nipy_spectral", s=0.05, edgecolors="none", vmin=0,
    ↪vmax=ftHE_avg.max())

ax[1].set_xlim(0, 1)
ax[1].set_ylim(0, 1)
ax[1].set_xlabel(r"$x$")
ax[1].set_ylabel(r"$y$")
ax[1].set_xticks([0, 0.5, 1])
ax[1].set_yticks([0, 0.5, 1])

counts, bins, patches = ax[2].hist(ftHE_avg, bins="auto", edgecolor='none',
    ↪density=True)
# Compute bin centers
```

```

bin_centers = 0.5 * (bins[:-1] + bins[1:])

# Normalize bin centers for colormap
norm = plt.Normalize(0, bin_centers.max())
colormap = cm.nipy_spectral # You can choose any colormap you like

# Apply color based on bin center (x position)
for center, patch in zip(bin_centers, patches):
    color = colormap(norm(center))
    patch.set_facecolor(color)

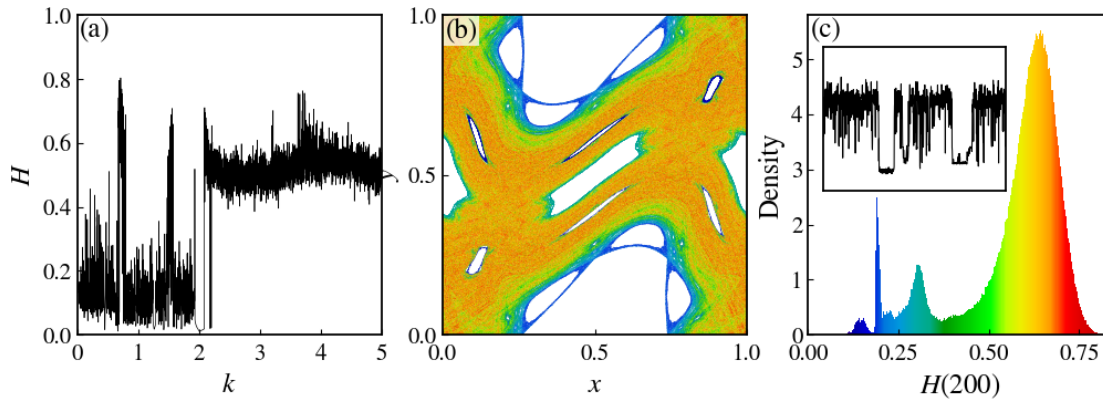
ax[2].set_xlim(bins[0], bins[-1])
ax[2].set_xlabel(r"$H(200)$")
ax[2].set_ylabel("Density")
ax[2].set_xlim(0, ftHE_avg.max())

ax_ins = ax[2].inset_axes([0.05, 0.45, 0.6, 0.45]) # [left, bottom, width,
↳height]
ii = np.arange(ftHE_avg.shape[0]) / 1e3
ax_ins.plot(ii, ftHE_avg, "k", lw=0.75)
ax_ins.set_xlim(1, 2)
ax_ins.set_ylim(0, 1)
ax_ins.set_yticks([])
ax_ins.set_xticks([])

xbox = 0.0095
ybox = 0.9318
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}
for i in range(3):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", bbox=bbox,
↳transform=ax[i].transAxes)
plt.subplots_adjust(left=0.059, bottom=0.15, right=0.995, top=0.975, wspace=0.
↳2, hspace=0.2)
plt.savefig("fig10.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



3 Periodic orbits and manifolds

```
[118]: ds = dds(model="standard map")
       k = 1.5
```

3.1 Period 1

The period-1 orbits can be found analytically. They are $(x, y) = (0, 0)$ and $(x, y) = (0.5, 0.0)$. Let us check their stability.

```
[119]: u = [0, 0]
       period = 1
       stability = ds.classify_stability(u, period, parameters=k)
       stability["classification"], stability["eigenvalues"]
```

```
[119]: ('saddle', array([3.18614066+0.j, 0.31385934+0.j]))
```

```
[120]: u = [0.5, 0]
       period = 1
       stability = ds.classify_stability(u, period, parameters=k)
       stability["classification"], stability["eigenvalues"]
```

```
[120]: ('elliptic (quasi-periodic)', array([0.25-0.96824584j, 0.25+0.96824584j]))
```

For the manifolds

```
[121]: saddle = [0, 0]
       n_points = 50000
       iter_time = 12
```

```
[122]: %%time
       wu_period1 = ds.manifold(saddle, period, parameters=k, n_points=n_points, u
       ↪ iter_time=iter_time, stability="unstable")
```

```
ws_period1 = ds.manifold(saddle, period, parameters=k, n_points=n_points,
↪iter_time=iter_time, stability="stable")
```

CPU times: user 577 ms, sys: 25.6 ms, total: 603 ms
Wall time: 490 ms

3.2 Period 2

Now for the period-2 orbit. We know that

```
[123]: periodic_orbit_center_p2 = [0.5, 0.5]
period = 2
stability = ds.classify_stability(periodic_orbit_center_p2, period,
↪parameters=k)
stability["classification"], stability["eigenvalues"]

[123]: ('elliptic (quasi-periodic)', array([-0.125+0.99215674j, -0.125-0.99215674j]))
```

```
[124]: x_range = (0.1, 0.3)
y_range = (0.3, 0.55)

period = 2
grid_size = 1000
tolerance = 2 / grid_size
x = np.linspace(x_range[0], x_range[1], grid_size)
y = np.linspace(y_range[0], y_range[1], grid_size)
X, Y = np.meshgrid(x, y)
grid_points = np.empty((grid_size, grid_size, 2), dtype=np.float64)
grid_points[:, :, 0] = X
grid_points[:, :, 1] = Y
```

```
[126]: %%time
periodic_orbit_saddle_p2 = ds.find_periodic_orbit(grid_points, period,
↪parameters=k, tolerance=tolerance, verbose=True, tolerance_decay_factor=0.5)
```

```
Iter 0: Δorbit=[0.19398951 0.38794242], Δbounds=[0.0006046 0.00075475],
tol=2.00e-03
Iter 1: Δorbit=[6.97569804e-07 1.17822834e-05], Δbounds=[0.0013954 0.00124525],
tol=1.00e-03
Iter 2: Δorbit=[1.30339304e-05 1.63665133e-07], Δbounds=[0.0002613 0.00024525],
tol=5.00e-04
Iter 3: Δorbit=[4.37991944e-06 4.97719348e-06], Δbounds=[0.0002387 0.00025475],
tol=2.50e-04
Iter 4: Δorbit=[2.51783269e-06 3.64189359e-06], Δbounds=[1.13033454e-05
4.75475475e-06], tol=1.25e-04
Iter 5: Δorbit=[5.20590344e-07 7.58901280e-06], Δbounds=[0.0001137 0.00012025],
tol=6.25e-05
Iter 6: Δorbit=[2.05217904e-06 9.98578301e-06], Δbounds=[1.85330511e-05
2.07930027e-05], tol=3.13e-05
```

Iter 7: $\Delta\text{orbit}=[5.93479191\text{e-}08 \ 4.74338684\text{e-}08]$, $\Delta\text{bounds}=[1.27169489\text{e-}05 \ 1.04569973\text{e-}05]$, $\text{tol}=1.56\text{e-}05$
 Iter 8: $\Delta\text{orbit}=[5.55501459\text{e-}09 \ 4.63855188\text{e-}09]$, $\Delta\text{bounds}=[2.90805114\text{e-}06 \ 5.16800271\text{e-}06]$, $\text{tol}=7.81\text{e-}06$
 Iter 9: $\Delta\text{orbit}=[1.13948853\text{e-}08 \ 1.26565990\text{e-}08]$, $\Delta\text{bounds}=[4.90444886\text{e-}06 \ 2.65484364\text{e-}06]$, $\text{tol}=3.91\text{e-}06$
 Iter 10: $\Delta\text{orbit}=[1.20663627\text{e-}08 \ 9.67360858\text{e-}10]$, $\Delta\text{bounds}=[1.43970534\text{e-}07 \ 1.25140636\text{e-}06]$, $\text{tol}=1.95\text{e-}06$
 Iter 11: $\Delta\text{orbit}=[1.75831010\text{e-}08 \ 2.26119669\text{e-}08]$, $\Delta\text{bounds}=[1.80915447\text{e-}06 \ 1.07375837\text{e-}06]$, $\text{tol}=9.77\text{e-}07$
 Iter 12: $\Delta\text{orbit}=[2.02341387\text{e-}08 \ 1.29956005\text{e-}08]$, $\Delta\text{bounds}=[1.98754065\text{e-}07 \ 9.71958682\text{e-}08]$, $\text{tol}=4.88\text{e-}07$
 Iter 13: $\Delta\text{orbit}=[5.86458457\text{e-}09 \ 1.11955997\text{e-}08]$, $\Delta\text{bounds}=[2.89527185\text{e-}07 \ 3.91085382\text{e-}07]$, $\text{tol}=2.44\text{e-}07$
 Iter 14: $\Delta\text{orbit}=[9.33819166\text{e-}09 \ 1.48005910\text{e-}08]$, $\Delta\text{bounds}=[4.53865599\text{e-}08 \ 6.16028015\text{e-}08]$, $\text{tol}=1.22\text{e-}07$
 Iter 15: $\Delta\text{orbit}=[5.59878710\text{e-}09 \ 3.51646018\text{e-}09]$, $\Delta\text{bounds}=[7.66837527\text{e-}08 \ 6.04675110\text{e-}08]$, $\text{tol}=6.10\text{e-}08$
 Iter 16: $\Delta\text{orbit}=[5.21837212\text{e-}09 \ 3.12760384\text{e-}09]$, $\Delta\text{bounds}=[7.20493995\text{e-}09 \ 5.67645264\text{e-}10]$, $\text{tol}=3.05\text{e-}08$
 Iter 17: $\Delta\text{orbit}=[9.95678762\text{e-}10 \ 3.13003140\text{e-}09]$, $\Delta\text{bounds}=[2.33126382\text{e-}08 \ 2.99499329\text{e-}08]$, $\text{tol}=1.53\text{e-}08$
 Iter 18: $\Delta\text{orbit}=[3.29292796\text{e-}09 \ 5.37633266\text{e-}09]$, $\Delta\text{bounds}=[2.73324702\text{e-}09 \ 3.77845560\text{e-}09]$, $\text{tol}=7.63\text{e-}09$
 Iter 19: $\Delta\text{orbit}=[6.32768810\text{e-}10 \ 3.99263123\text{e-}10]$, $\Delta\text{bounds}=[4.89614749\text{e-}09 \ 3.85093896\text{e-}09]$, $\text{tol}=3.81\text{e-}09$
 Iter 20: $\Delta\text{orbit}=[5.62221686\text{e-}10 \ 3.48825469\text{e-}10]$, $\Delta\text{bounds}=[2.13964624\text{e-}10 \ 9.25814980\text{e-}12]$, $\text{tol}=1.91\text{e-}09$
 Iter 21: $\Delta\text{orbit}=[1.28868194\text{e-}10 \ 3.35720673\text{e-}10]$, $\Delta\text{bounds}=[1.69338399\text{e-}09 \ 1.89809046\text{e-}09]$, $\text{tol}=9.54\text{e-}10$
 Iter 22: $\Delta\text{orbit}=[3.75867615\text{e-}10 \ 5.57253244\text{e-}10]$, $\Delta\text{bounds}=[5.1507576\text{e-}11 \ 1.7791324\text{e-}12]$, $\text{tol}=4.77\text{e-}10$
 Iter 23: $\Delta\text{orbit}=[3.14295534\text{e-}11 \ 8.41683945\text{e-}11]$, $\Delta\text{bounds}=[4.25329605\text{e-}10 \ 4.75057993\text{e-}10]$, $\text{tol}=2.38\text{e-}10$
 Iter 24: $\Delta\text{orbit}=[9.41929590\text{e-}11 \ 1.40366163\text{e-}10]$, $\Delta\text{bounds}=[1.40543688\text{e-}11 \ 1.12737597\text{e-}12]$, $\text{tol}=1.19\text{e-}10$
 Iter 25: $\Delta\text{orbit}=[8.37774294\text{e-}12 \ 2.14752660\text{e-}11]$, $\Delta\text{bounds}=[1.05999043\text{e-}10 \ 1.18081933\text{e-}10]$, $\text{tol}=5.96\text{e-}11$
 Iter 26: $\Delta\text{orbit}=[2.36753395\text{e-}11 \ 3.52891050\text{e-}11]$, $\Delta\text{bounds}=[3.20959925\text{e-}12 \ 1.21380683\text{e-}12]$, $\text{tol}=2.98\text{e-}11$
 Iter 27: $\Delta\text{orbit}=[1.98430161\text{e-}12 \ 5.39102096\text{e-}12]$, $\Delta\text{bounds}=[2.65927280\text{e-}11 \ 2.85884649\text{e-}11]$, $\text{tol}=1.49\text{e-}11$
 Iter 28: $\Delta\text{orbit}=[5.86569682\text{e-}12 \ 8.86546392\text{e-}12]$, $\Delta\text{bounds}=[8.84126106\text{e-}13 \ 1.07913678\text{e-}12]$, $\text{tol}=7.45\text{e-}12$
 Iter 29: $\Delta\text{orbit}=[3.45362627\text{e-}13 \ 1.20320420\text{e-}12]$, $\Delta\text{bounds}=[6.63460953\text{e-}12 \ 6.86078971\text{e-}12]$, $\text{tol}=3.73\text{e-}12$
 Iter 30: $\Delta\text{orbit}=[1.40709666\text{e-}12 \ 2.18519647\text{e-}12]$, $\Delta\text{bounds}=[1.99673611\text{e-}13 \ 6.00464123\text{e-}13]$, $\text{tol}=1.86\text{e-}12$

```

Iter 31: Δorbit=[5.34572386e-14 1.66144876e-13], Δbounds=[1.66294756e-12
1.71174186e-12], tol=9.31e-13
Iter 32: Δorbit=[3.48970852e-13 5.42954570e-13], Δbounds=[5.24857935e-14
1.46604950e-13], tol=4.66e-13
Iter 33: Δorbit=[1.15463195e-14 4.32986980e-14], Δbounds=[4.14834833e-13
4.28934666e-13], tol=2.33e-13
Iter 34: Δorbit=[8.74578188e-14 1.36335387e-13], Δbounds=[1.34336986e-14
3.65818487e-14], tol=1.16e-13
Iter 35: Δorbit=[2.80331314e-15 1.09356968e-14], Δbounds=[1.03500541e-13
1.07136522e-13], tol=5.82e-14
Iter 36: Δorbit=[2.18713936e-14 3.40838469e-14], Δbounds=[3.21964677e-15
9.15933995e-15], tol=2.91e-14
Iter 37: Δorbit=[7.49400542e-16 2.72004641e-15], Δbounds=[2.59514632e-14
2.68118860e-14], tol=1.46e-14
Iter 38: Δorbit=[5.41233725e-15 8.60422844e-15], Δbounds=[9.15933995e-16
2.27595720e-15], tol=7.28e-15
Iter 39: Δorbit=[2.22044605e-16 8.88178420e-16], Δbounds=[6.46704912e-15
6.77236045e-15], tol=3.64e-15
Iter 40: Δorbit=[1.38777878e-15 2.27595720e-15], Δbounds=[1.11022302e-16
7.21644966e-16], tol=1.82e-15
Iter 41: Δorbit=[1.11022302e-16 3.88578059e-16], Δbounds=[1.72084569e-15
1.49880108e-15], tol=9.09e-16
Iter 42: Δorbit=[3.05311332e-16 6.66133815e-16], Δbounds=[1.38777878e-16
1.11022302e-16], tol=4.55e-16
Converged after 42 iterations
CPU times: user 9.76 s, sys: 169 ms, total: 9.93 s
Wall time: 1.78 s

```

```

[127]: periodic_orbit_saddle_p2, ds.classify_stability(periodic_orbit_saddle_p2,
↳period, parameters=k)

```

```

[127]: (array([0.19397649, 0.38795298]),
{'classification': 'saddle',
 'eigenvalues': array([4.09176343+0.j, 0.24439341+0.j]),
 'eigenvectors': array([[ 0.89240544+0.j, -0.69908845+0.j],
 [ 0.45123445+0.j, 0.7150352 +0.j]])})

```

```

[128]: %%time
n_points = 50000
iter_time = 17
wu_period2 = ds.manifold(periodic_orbit_saddle_p2, period, parameters=k,
↳n_points=n_points, iter_time=iter_time, stability="unstable")
ws_period2 = ds.manifold(periodic_orbit_saddle_p2, period, parameters=k,
↳n_points=n_points, iter_time=iter_time, stability="stable")

```

```

CPU times: user 214 ms, sys: 9.35 ms, total: 223 ms
Wall time: 42.9 ms

```

3.3 Period 3

There two period 3 period orbits. Let us find the lower one first.

3.3.1 Lower period 3

For the center.

```
[130]: # Define the symmetry line
symmetry_line = lambda v, parameters: 0.5 * np.ones_like(v)
# Define the type of the function, i.e.,  $x = g(y)$ 
axis = 1
# Define the period
period = 3
# Define the range of the initial search
y_range = (0.2, 0.4)
# Define the number of points in the range
num_points = 10000
# Define the initial conditions
points = np.linspace(y_range[0], y_range[1], num_points)
tolerance = 2 / num_points

[132]: %%time
periodic_orbit_center_period3_lower = ds.find_periodic_orbit(points, period,
↳ parameters=k, tolerance=tolerance, symmetry_line=symmetry_line, axis=axis,
↳ verbose=True, tolerance_decay_factor=0.8)

Iter 0: Δorbit=[0.5          0.38569857], Δbounds=[0.0004  0.00036], tol=2.00e-04
Iter 1: Δorbit=[0.00000000e+00  1.60214241e-06], Δbounds=[0.00032  0.00028364],
tol=1.60e-04
Iter 2: Δorbit=[0. 0.], Δbounds=[0.000256  0.00022692], tol=1.28e-04
Iter 3: Δorbit=[0.00000000e+00  1.13473509e-08], Δbounds=[0.0002048  0.00018152],
tol=1.02e-04
Iter 4: Δorbit=[0.00000000e+00  5.55111512e-17], Δbounds=[0.00016384  0.00014523],
tol=8.19e-05
Iter 5: Δorbit=[0. 0.], Δbounds=[0.00013107  0.00011618], tol=6.55e-05
Iter 6: Δorbit=[0. 0.], Δbounds=[1.0485760e-04  9.2947534e-05], tol=5.24e-05
Iter 7: Δorbit=[0. 0.], Δbounds=[8.3886080e-05  7.4358005e-05], tol=4.19e-05
Iter 8: Δorbit=[0. 0.], Δbounds=[6.71088640e-05  5.94864062e-05], tol=3.36e-05
Iter 9: Δorbit=[0. 0.], Δbounds=[5.36870912e-05  4.75891248e-05], tol=2.68e-05
Iter 10: Δorbit=[0. 0.], Δbounds=[4.29496730e-05  3.80712998e-05], tol=2.15e-05
Iter 11: Δorbit=[0.00000000e+00  5.55111512e-17], Δbounds=[3.43597384e-05
3.04570399e-05], tol=1.72e-05
Iter 12: Δorbit=[0.00000000e+00  1.52300433e-09], Δbounds=[2.74877907e-05
2.43625859e-05], tol=1.37e-05
Iter 13: Δorbit=[0. 0.], Δbounds=[2.19902326e-05  1.94928178e-05], tol=1.10e-05
Iter 14: Δorbit=[0. 0.], Δbounds=[1.75921860e-05  1.55939724e-05], tol=8.80e-06
Iter 15: Δorbit=[0. 0.], Δbounds=[1.40737488e-05  1.24752068e-05], tol=7.04e-06
Iter 16: Δorbit=[0. 0.], Δbounds=[1.12589991e-05  9.98016249e-06], tol=5.63e-06
```

Iter 17: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[9.00719925\text{e-}06\ 7.98413029\text{e-}06]$, $\text{tol}=4.50\text{e-}06$
 Iter 18: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[7.2057594\text{e-}06\ 6.3873042\text{e-}06]$, $\text{tol}=3.60\text{e-}06$
 Iter 19: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[5.76460752\text{e-}06\ 5.10984337\text{e-}06]$, $\text{tol}=2.88\text{e-}06$
 Iter 20: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[4.61168602\text{e-}06\ 4.08787469\text{e-}06]$, $\text{tol}=2.31\text{e-}06$
 Iter 21: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.68934881\text{e-}06\ 3.27029975\text{e-}06]$, $\text{tol}=1.84\text{e-}06$
 Iter 22: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.95147905\text{e-}06\ 2.61623980\text{e-}06]$, $\text{tol}=1.48\text{e-}06$
 Iter 23: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.36118324\text{e-}06\ 2.09299184\text{e-}06]$, $\text{tol}=1.18\text{e-}06$
 Iter 24: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.88894659\text{e-}06\ 1.67439347\text{e-}06]$, $\text{tol}=9.44\text{e-}07$
 Iter 25: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.51115727\text{e-}06\ 1.33951478\text{e-}06]$, $\text{tol}=7.56\text{e-}07$
 Iter 26: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 6.69824196\text{e-}11]$, $\Delta\text{bounds}=[1.20892582\text{e-}06\ 1.07147786\text{e-}06]$, $\text{tol}=6.04\text{e-}07$
 Iter 27: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[9.67140656\text{e-}07\ 8.57303191\text{e-}07]$, $\text{tol}=4.84\text{e-}07$
 Iter 28: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[7.73712525\text{e-}07\ 6.85830159\text{e-}07]$, $\text{tol}=3.87\text{e-}07$
 Iter 29: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[6.18970020\text{e-}07\ 5.48665398\text{e-}07]$, $\text{tol}=3.09\text{e-}07$
 Iter 30: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[4.95176016\text{e-}07\ 4.38932188\text{e-}07]$, $\text{tol}=2.48\text{e-}07$
 Iter 31: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.96140813\text{e-}07\ 3.51145764\text{e-}07]$, $\text{tol}=1.98\text{e-}07$
 Iter 32: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[3.1691265\text{e-}07\ 2.8091661\text{e-}07]$, $\text{tol}=1.58\text{e-}07$
 Iter 33: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.53530120\text{e-}07\ 2.24733288\text{e-}07]$, $\text{tol}=1.27\text{e-}07$
 Iter 34: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.12378995\text{e-}11]$, $\Delta\text{bounds}=[2.02824096\text{e-}07\ 1.79764155\text{e-}07]$, $\text{tol}=1.01\text{e-}07$
 Iter 35: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.62259277\text{e-}07\ 1.43831608\text{e-}07]$, $\text{tol}=8.11\text{e-}08$
 Iter 36: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.29807421\text{e-}07\ 1.15063207\text{e-}07]$, $\text{tol}=6.49\text{e-}08$
 Iter 37: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.03845937\text{e-}07\ 9.20507789\text{e-}08]$, $\text{tol}=5.19\text{e-}08$
 Iter 38: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[8.30767497\text{e-}08\ 7.36406013\text{e-}08]$, $\text{tol}=4.15\text{e-}08$
 Iter 39: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[6.64613998\text{e-}08\ 5.89124833\text{e-}08]$, $\text{tol}=3.32\text{e-}08$
 Iter 40: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[5.31691198\text{e-}08\ 4.71299865\text{e-}08]$, $\text{tol}=2.66\text{e-}08$
 Iter 41: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 2.35672593\text{e-}12]$, $\Delta\text{bounds}=[4.25352959\text{e-}08\ 3.76992756\text{e-}08]$, $\text{tol}=2.13\text{e-}08$
 Iter 42: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.40282367\text{e-}08\ 3.01636744\text{e-}08]$, $\text{tol}=1.70\text{e-}08$
 Iter 43: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.72225893\text{e-}08\ 2.41305036\text{e-}08]$, $\text{tol}=1.36\text{e-}08$
 Iter 44: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.17780715\text{e-}08\ 1.93044475\text{e-}08]$, $\text{tol}=1.09\text{e-}08$
 Iter 45: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.74224572\text{e-}08\ 1.54435534\text{e-}08]$, $\text{tol}=8.71\text{e-}09$
 Iter 46: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.39379657\text{e-}08\ 1.23548433\text{e-}08]$, $\text{tol}=6.97\text{e-}09$
 Iter 47: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.11503726\text{e-}08\ 9.88387450\text{e-}09]$, $\text{tol}=5.58\text{e-}09$
 Iter 48: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[8.92029806\text{e-}09\ 7.90709964\text{e-}09]$, $\text{tol}=4.46\text{e-}09$
 Iter 49: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[7.13623849\text{e-}09\ 6.32567976\text{e-}09]$, $\text{tol}=3.57\text{e-}09$
 Iter 50: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[5.70899078\text{e-}09\ 5.06054365\text{e-}09]$, $\text{tol}=2.85\text{e-}09$
 Iter 51: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[4.56719262\text{e-}09\ 4.04843503\text{e-}09]$, $\text{tol}=2.28\text{e-}09$
 Iter 52: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.65375413\text{e-}09\ 3.23874794\text{e-}09]$, $\text{tol}=1.83\text{e-}09$
 Iter 53: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.92300323\text{e-}09\ 2.59099842\text{e-}09]$, $\text{tol}=1.46\text{e-}09$
 Iter 54: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.33840258\text{e-}09\ 2.07279871\text{e-}09]$, $\text{tol}=1.17\text{e-}09$
 Iter 55: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.03583808\text{e-}13]$, $\Delta\text{bounds}=[1.87072208\text{e-}09\ 1.65803171\text{e-}09]$, $\text{tol}=9.35\text{e-}10$

Iter 56: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.49657770\text{e-}09\ 1.32661238\text{e-}09]$, $\text{tol}=7.48\text{e-}10$
 Iter 57: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.19726212\text{e-}09\ 1.06127085\text{e-}09]$, $\text{tol}=5.99\text{e-}10$
 Iter 58: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[9.57809665\text{e-}10\ 8.49018633\text{e-}10]$, $\text{tol}=4.79\text{e-}10$
 Iter 59: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[7.66247732\text{e-}10\ 6.79214573\text{e-}10]$, $\text{tol}=3.83\text{e-}10$
 Iter 60: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[6.12998263\text{e-}10\ 5.43371792\text{e-}10]$, $\text{tol}=3.06\text{e-}10$
 Iter 61: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[4.90398611\text{e-}10\ 4.34697389\text{e-}10]$, $\text{tol}=2.45\text{e-}10$
 Iter 62: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[3.92318844\text{e-}10\ 3.47757934\text{e-}10]$, $\text{tol}=1.96\text{e-}10$
 Iter 63: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.13855109\text{e-}10\ 2.78206347\text{e-}10]$, $\text{tol}=1.57\text{e-}10$
 Iter 64: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.51084042\text{e-}10\ 2.22565077\text{e-}10]$, $\text{tol}=1.26\text{e-}10$
 Iter 65: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.00867267\text{e-}10\ 1.78052018\text{e-}10]$, $\text{tol}=1.00\text{e-}10$
 Iter 66: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.60693847\text{e-}10\ 1.42441614\text{e-}10]$, $\text{tol}=8.03\text{e-}11$
 Iter 67: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.28555000\text{e-}10\ 1.13953291\text{e-}10]$, $\text{tol}=6.43\text{e-}11$
 Iter 68: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.02844011\text{e-}10\ 9.11626330\text{e-}11]$, $\text{tol}=5.14\text{e-}11$
 Iter 69: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[8.22752422\text{e-}11\ 7.29301064\text{e-}11]$, $\text{tol}=4.11\text{e-}11$
 Iter 70: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[6.58201826\text{e-}11\ 5.83439963\text{e-}11]$, $\text{tol}=3.29\text{e-}11$
 Iter 71: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[5.26561017\text{e-}11\ 4.66752192\text{e-}11]$, $\text{tol}=2.63\text{e-}11$
 Iter 72: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 2.33146835\text{e-}15]$, $\Delta\text{bounds}=[4.21249702\text{e-}11\ 3.73355791\text{e-}11]$, $\text{tol}=2.11\text{e-}11$
 Iter 73: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[3.36999317\text{e-}11\ 2.98726599\text{e-}11]$, $\text{tol}=1.68\text{e-}11$
 Iter 74: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.69599898\text{e-}11\ 2.38977726\text{e-}11]$, $\text{tol}=1.35\text{e-}11$
 Iter 75: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.15679141\text{e-}11\ 1.91181515\text{e-}11]$, $\text{tol}=1.08\text{e-}11$
 Iter 76: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.72544201\text{e-}11\ 1.52946544\text{e-}11]$, $\text{tol}=8.63\text{e-}12$
 Iter 77: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.38034584\text{e-}11\ 1.22356569\text{e-}11]$, $\text{tol}=6.90\text{e-}12$
 Iter 78: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.10427778\text{e-}11\ 9.78850334\text{e-}12]$, $\text{tol}=5.52\text{e-}12$
 Iter 79: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}16]$, $\Delta\text{bounds}=[8.83426665\text{e-}12\ 7.82984788\text{e-}12]$, $\text{tol}=4.42\text{e-}12$
 Iter 80: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[7.06740222\text{e-}12\ 6.26465546\text{e-}12]$, $\text{tol}=3.53\text{e-}12$
 Iter 81: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[5.65392178\text{e-}12\ 5.01176878\text{e-}12]$, $\text{tol}=2.83\text{e-}12$
 Iter 82: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[4.52310411\text{e-}12\ 4.00934841\text{e-}12]$, $\text{tol}=2.26\text{e-}12$
 Iter 83: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.61849439\text{e-}12\ 3.20754534\text{e-}12]$, $\text{tol}=1.81\text{e-}12$
 Iter 84: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.89479551\text{e-}12\ 2.56566990\text{e-}12]$, $\text{tol}=1.45\text{e-}12$

Iter 85: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.31586972\text{e-}12\ 2.05285788\text{e-}12]$, $\text{tol}=1.16\text{e-}12$
 Iter 86: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.85268467\text{e-}12\ 1.64240843\text{e-}12]$, $\text{tol}=9.26\text{e-}13$
 Iter 87: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.66533454\text{e-}16]$, $\Delta\text{bounds}=[1.48214774\text{e-}12\ 1.31378242\text{e-}12]$, $\text{tol}=7.41\text{e-}13$
 Iter 88: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.18571819\text{e-}12\ 1.05104814\text{e-}12]$, $\text{tol}=5.93\text{e-}13$
 Iter 89: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[9.48574552\text{e-}13\ 8.40771897\text{e-}13]$, $\text{tol}=4.74\text{e-}13$
 Iter 90: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[7.58892948\text{e-}13\ 6.72684131\text{e-}13]$, $\text{tol}=3.79\text{e-}13$
 Iter 91: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[6.07069950\text{e-}13\ 5.38236122\text{e-}13]$, $\text{tol}=3.04\text{e-}13$
 Iter 92: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[4.85667062\text{e-}13\ 4.30544489\text{e-}13]$, $\text{tol}=2.43\text{e-}13$
 Iter 93: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.88578059\text{e-}13\ 3.44502205\text{e-}13]$, $\text{tol}=1.94\text{e-}13$
 Iter 94: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.10862447\text{e-}13\ 2.75612866\text{e-}13]$, $\text{tol}=1.55\text{e-}13$
 Iter 95: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.48689958\text{e-}13\ 2.20490293\text{e-}13]$, $\text{tol}=1.24\text{e-}13$
 Iter 96: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.98951966\text{e-}13\ 1.76358927\text{e-}13]$, $\text{tol}=9.95\text{e-}14$
 Iter 97: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.59150471\text{e-}13\ 1.40998324\text{e-}13]$, $\text{tol}=7.96\text{e-}14$
 Iter 98: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.27287070\text{e-}13\ 1.12965193\text{e-}13]$, $\text{tol}=6.37\text{e-}14$
 Iter 99: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.01862963\text{e-}13\ 9.03166431\text{e-}14]$, $\text{tol}=5.09\text{e-}14$
 Iter 100: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[8.14903700\text{e-}14\ 7.22755189\text{e-}14]$, $\text{tol}=4.07\text{e-}14$
 Iter 101: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[6.52256027\text{e-}14\ 5.78981307\text{e-}14]$, $\text{tol}=3.26\text{e-}14$
 Iter 102: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[5.21804822\text{e-}14\ 4.64073224\text{e-}14]$, $\text{tol}=2.61\text{e-}14$
 Iter 103: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[4.17443857\text{e-}14\ 3.69704267\text{e-}14]$, $\text{tol}=2.09\text{e-}14$
 Iter 104: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[3.33622019\text{e-}14\ 2.97539771\text{e-}14]$, $\text{tol}=1.67\text{e-}14$
 Iter 105: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.66453526\text{e-}14\ 2.35922393\text{e-}14]$, $\text{tol}=1.33\text{e-}14$
 Iter 106: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.13162821\text{e-}14\ 1.89293026\text{e-}14]$, $\text{tol}=1.07\text{e-}14$
 Iter 107: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[1.70974346\text{e-}14\ 1.52100554\text{e-}14]$, $\text{tol}=8.54\text{e-}15$
 Iter 108: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.37112544\text{e-}14\ 1.21569421\text{e-}14]$, $\text{tol}=6.84\text{e-}15$
 Iter 109: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[1.09356968\text{e-}14\ 9.76996262\text{e-}15]$, $\text{tol}=5.47\text{e-}15$
 Iter 110: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[8.71525074\text{e-}15\ 7.88258347\text{e-}15]$, $\text{tol}=4.37\text{e-}15$
 Iter 111: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[7.04991621\text{e-}15\ 6.16173779\text{e-}15]$, $\text{tol}=3.50\text{e-}15$
 Iter 112: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[5.55111512\text{e-}15\ 5.10702591\text{e-}15]$, $\text{tol}=2.80\text{e-}15$
 Iter 113: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[4.44089210\text{e-}15\ 3.99680289\text{e-}15]$, $\text{tol}=2.24\text{e-}15$
 Iter 114: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[3.55271368\text{e-}15\ 3.16413562\text{e-}15]$, $\text{tol}=1.79\text{e-}15$
 Iter 115: $\Delta\text{orbit}=[0.0.]$, $\Delta\text{bounds}=[2.88657986\text{e-}15\ 2.55351296\text{e-}15]$, $\text{tol}=1.43\text{e-}15$
 Iter 116: $\Delta\text{orbit}=[0.00000000\text{e+}00\ 5.55111512\text{e-}17]$, $\Delta\text{bounds}=[2.2759572\text{e-}15$

```

2.0539126e-15], tol=1.15e-15
Iter 117: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.83186799e-15
1.60982339e-15], tol=9.17e-16
Iter 118: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.49880108e-15
1.27675648e-15], tol=7.34e-16
Iter 119: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.16573418e-15
1.11022302e-15], tol=5.87e-16
Iter 120: Δorbit=[0.00000000e+00 1.66533454e-16], Δbounds=[8.88178420e-16
8.32667268e-16], tol=4.70e-16
Converged at iteration 120
CPU times: user 223 ms, sys: 4.4 ms, total: 227 ms
Wall time: 228 ms

```

```

[134]: periodic_orbit_center_period3_lower, ds.
       ↪ classify_stability(periodic_orbit_center_period3_lower, period, parameters=k)

```

```

[134]: (array([0.5          , 0.38569696]),
       {'classification': 'elliptic (quasi-periodic)',
        'eigenvalues': array([-0.93105758-0.36487228j, -0.93105758+0.36487228j]),
        'eigenvectors': array([[ -0.31198762+0.06967997j, -0.31198762-0.06967997j],
                                [ 0.94752753+0.j          , 0.94752753+0.j          ]])})

```

For the saddle

```

[135]: # Define the symmetry line
symmetry_line = lambda v, parameters: 0.0 * np.ones_like(v)
# Define the type of the function, i.e.,  $x = g(y)$ 
axis = 1
# Define the period
period = 3
# Define the range of the initial search
y_range = (0.2, 0.3)
# Define the number of points in the range
num_points = 10000
# Define the initial conditions
points = np.linspace(y_range[0], y_range[1], num_points)
tolerance = 2 / num_points

```

```

[137]: %%time
periodic_orbit_saddle_period3_lower = ds.find_periodic_orbit(points, period,
       ↪ parameters=k, tolerance=tolerance, symmetry_line=symmetry_line, axis=axis,
       ↪ verbose=True, tolerance_decay_factor=0.8)

```

```

Iter 0: Δorbit=[0.          0.25381538], Δbounds=[0.0004      0.00033999],
tol=2.00e-04
Iter 1: Δorbit=[0.00000000e+00 9.11275046e-06], Δbounds=[0.00032      0.00026407],
tol=1.60e-04
Iter 2: Δorbit=[0.00000000e+00 5.61195081e-06], Δbounds=[0.000256      0.00021126],
tol=1.28e-04

```

Iter 3: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.46865611\text{e}-06]$, $\Delta\text{bounds}=[0.0002048 \ 0.00016901]$,
 tol=1.02e-04
 Iter 4: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.57488714\text{e}-06]$, $\Delta\text{bounds}=[0.00016384 \ 0.00013521]$,
 tol=8.19e-05
 Iter 5: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.86667714\text{e}-06]$, $\Delta\text{bounds}=[0.00013107 \ 0.00010815]$,
 tol=6.55e-05
 Iter 6: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.29305393\text{e}-06]$, $\Delta\text{bounds}=[1.04857600\text{e}-04 \ 8.65239849\text{e}-05]$,
 tol=5.24e-05
 Iter 7: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.83449193\text{e}-06]$, $\Delta\text{bounds}=[8.38860800\text{e}-05 \ 6.92187978\text{e}-05]$,
 tol=4.19e-05
 Iter 8: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.46758527\text{e}-06]$, $\Delta\text{bounds}=[6.71088640\text{e}-05 \ 5.53751044\text{e}-05]$,
 tol=3.36e-05
 Iter 9: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.17130059\text{e}-06]$, $\Delta\text{bounds}=[5.36870912\text{e}-05 \ 4.43056104\text{e}-05]$,
 tol=2.68e-05
 Iter 10: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.41588379\text{e}-07]$, $\Delta\text{bounds}=[4.2949673\text{e}-05 \ 3.5443552\text{e}-05]$,
 tol=2.15e-05
 Iter 11: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.49706094\text{e}-07]$, $\Delta\text{bounds}=[3.43597384\text{e}-05 \ 2.83550002\text{e}-05]$,
 tol=1.72e-05
 Iter 12: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.99768231\text{e}-07]$, $\Delta\text{bounds}=[2.74877907\text{e}-05 \ 2.26839733\text{e}-05]$,
 tol=1.37e-05
 Iter 13: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.8208264\text{e}-07]$, $\Delta\text{bounds}=[2.19902326\text{e}-05 \ 1.81471832\text{e}-05]$,
 tol=1.10e-05
 Iter 14: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.83851309\text{e}-07]$, $\Delta\text{bounds}=[1.75921860\text{e}-05 \ 1.45177458\text{e}-05]$,
 tol=8.80e-06
 Iter 15: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.07806991\text{e}-07]$, $\Delta\text{bounds}=[1.40737488\text{e}-05 \ 1.16127448\text{e}-05]$,
 tol=7.04e-06
 Iter 16: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.46214812\text{e}-07]$, $\Delta\text{bounds}=[1.12589991\text{e}-05 \ 9.29044196\text{e}-06]$,
 tol=5.63e-06
 Iter 17: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.96977067\text{e}-07]$, $\Delta\text{bounds}=[9.00719925\text{e}-06 \ 7.43231185\text{e}-06]$,
 tol=4.50e-06
 Iter 18: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.57580769\text{e}-07]$, $\Delta\text{bounds}=[7.20575940\text{e}-06 \ 5.94585655\text{e}-06]$,
 tol=3.60e-06
 Iter 19: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.26064765\text{e}-07]$, $\Delta\text{bounds}=[5.76460752\text{e}-06 \ 4.75668404\text{e}-06]$,
 tol=2.88e-06
 Iter 20: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.00613929\text{e}-07]$, $\Delta\text{bounds}=[4.61168602\text{e}-06 \ 3.80582315\text{e}-06]$,
 tol=2.31e-06
 Iter 21: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 8.08818302\text{e}-08]$, $\Delta\text{bounds}=[3.68934881\text{e}-06 \ 3.04457790\text{e}-06]$,
 tol=1.84e-06
 Iter 22: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.43992624\text{e}-08]$, $\Delta\text{bounds}=[2.95147905\text{e}-06 \ 2.43567598\text{e}-06]$,
 tol=1.48e-06
 Iter 23: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.16414949\text{e}-08]$, $\Delta\text{bounds}=[2.36118324\text{e}-06 \ 1.94829487\text{e}-06]$,
 tol=1.18e-06
 Iter 24: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.13079821\text{e}-08]$, $\Delta\text{bounds}=[1.88894659\text{e}-06 \ 1.55867758\text{e}-06]$,
 tol=9.44e-07
 Iter 25: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.30472695\text{e}-08]$, $\Delta\text{bounds}=[1.51115727\text{e}-06 \ 1.24693500\text{e}-06]$,
 tol=7.56e-07
 Iter 26: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.63753128\text{e}-08]$, $\Delta\text{bounds}=[1.20892582\text{e}-06 \ 9.97673905\text{e}-07]$,
 tol=6.04e-07

Iter 27: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.12026908\text{e}-08]$, $\Delta\text{bounds}=[9.67140656\text{e}-07 \ 7.98117794\text{e}-07]$, $\text{tol}=4.84\text{e}-07$
 Iter 28: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.68818795\text{e}-08]$, $\Delta\text{bounds}=[7.73712525\text{e}-07 \ 6.38497849\text{e}-07]$, $\text{tol}=3.87\text{e}-07$
 Iter 29: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.35375082\text{e}-08]$, $\Delta\text{bounds}=[6.18970020\text{e}-07 \ 5.10733811\text{e}-07]$, $\text{tol}=3.09\text{e}-07$
 Iter 30: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.08286397\text{e}-08]$, $\Delta\text{bounds}=[4.95176016\text{e}-07 \ 4.08597977\text{e}-07]$, $\text{tol}=2.48\text{e}-07$
 Iter 31: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 8.64271149\text{e}-09]$, $\Delta\text{bounds}=[3.96140813\text{e}-07 \ 3.26917393\text{e}-07]$, $\text{tol}=1.98\text{e}-07$
 Iter 32: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.94768937\text{e}-09]$, $\Delta\text{bounds}=[3.16912650\text{e}-07 \ 2.61527305\text{e}-07]$, $\text{tol}=1.58\text{e}-07$
 Iter 33: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.5318557\text{e}-09]$, $\Delta\text{bounds}=[2.53530120\text{e}-07 \ 2.09222964\text{e}-07]$, $\text{tol}=1.27\text{e}-07$
 Iter 34: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.43597042\text{e}-09]$, $\Delta\text{bounds}=[2.02824096\text{e}-07 \ 1.67357257\text{e}-07]$, $\text{tol}=1.01\text{e}-07$
 Iter 35: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.54832869\text{e}-09]$, $\Delta\text{bounds}=[1.62259277\text{e}-07 \ 1.33889385\text{e}-07]$, $\text{tol}=8.11\text{e}-08$
 Iter 36: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.83204371\text{e}-09]$, $\Delta\text{bounds}=[1.29807421\text{e}-07 \ 1.07124291\text{e}-07]$, $\text{tol}=6.49\text{e}-08$
 Iter 37: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.27126207\text{e}-09]$, $\Delta\text{bounds}=[1.03845937\text{e}-07 \ 8.56865539\text{e}-08]$, $\text{tol}=5.19\text{e}-08$
 Iter 38: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.81673665\text{e}-09]$, $\Delta\text{bounds}=[8.30767497\text{e}-08 \ 6.85514263\text{e}-08]$, $\text{tol}=4.15\text{e}-08$
 Iter 39: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.45343554\text{e}-09]$, $\Delta\text{bounds}=[6.64613998\text{e}-08 \ 5.48407710\text{e}-08]$, $\text{tol}=3.32\text{e}-08$
 Iter 40: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.16274063\text{e}-09]$, $\Delta\text{bounds}=[5.31691198\text{e}-08 \ 4.38726795\text{e}-08]$, $\text{tol}=2.66\text{e}-08$
 Iter 41: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.30193811\text{e}-10]$, $\Delta\text{bounds}=[4.25352959\text{e}-08 \ 3.50981330\text{e}-08]$, $\text{tol}=2.13\text{e}-08$
 Iter 42: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.44154849\text{e}-10]$, $\Delta\text{bounds}=[3.40282367\text{e}-08 \ 2.80785082\text{e}-08]$, $\text{tol}=1.70\text{e}-08$
 Iter 43: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.93919858\text{e}-10]$, $\Delta\text{bounds}=[2.72225894\text{e}-08 \ 2.24656144\text{e}-08]$, $\text{tol}=1.36\text{e}-08$
 Iter 44: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.7744203\text{e}-10]$, $\Delta\text{bounds}=[2.17780715\text{e}-08 \ 1.79720158\text{e}-08]$, $\text{tol}=1.09\text{e}-08$
 Iter 45: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.80146192\text{e}-10]$, $\Delta\text{bounds}=[1.74224572\text{e}-08 \ 1.43776932\text{e}-08]$, $\text{tol}=8.71\text{e}-09$
 Iter 46: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.04837544\text{e}-10]$, $\Delta\text{bounds}=[1.39379657\text{e}-08 \ 1.15007031\text{e}-08]$, $\text{tol}=6.97\text{e}-09$
 Iter 47: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.43839282\text{e}-10]$, $\Delta\text{bounds}=[1.11503726\text{e}-08 \ 9.20080850\text{e}-09]$, $\text{tol}=5.58\text{e}-09$
 Iter 48: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.95076677\text{e}-10]$, $\Delta\text{bounds}=[8.92029808\text{e}-09 \ 7.36060501\text{e}-09]$, $\text{tol}=4.46\text{e}-09$
 Iter 49: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.55692348\text{e}-10]$, $\Delta\text{bounds}=[7.13623846\text{e}-09 \ 5.88922727\text{e}-09]$, $\text{tol}=3.57\text{e}-09$
 Iter 50: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.25158606\text{e}-10]$, $\Delta\text{bounds}=[5.70899077\text{e}-09 \ 4.71125589\text{e}-09]$, $\text{tol}=2.85\text{e}-09$

Iter 51: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.96530636\text{e}-11]$, $\Delta\text{bounds}=[4.56719262\text{e}-09 \ 3.76902609\text{e}-09]$, $\text{tol}=2.28\text{e}-09$
 Iter 52: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.99112998\text{e}-11]$, $\Delta\text{bounds}=[3.65375409\text{e}-09 \ 3.01484027\text{e}-09]$, $\text{tol}=1.83\text{e}-09$
 Iter 53: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.39210351\text{e}-11]$, $\Delta\text{bounds}=[2.92300327\text{e}-09 \ 2.41193671\text{e}-09]$, $\text{tol}=1.46\text{e}-09$
 Iter 54: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.11382048\text{e}-11]$, $\Delta\text{bounds}=[2.33840262\text{e}-09 \ 1.92953842\text{e}-09]$, $\text{tol}=1.17\text{e}-09$
 Iter 55: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.08137968\text{e}-11]$, $\Delta\text{bounds}=[1.87072210\text{e}-09 \ 1.54382568\text{e}-09]$, $\text{tol}=9.35\text{e}-10$
 Iter 56: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.28096439\text{e}-11]$, $\Delta\text{bounds}=[1.49657768\text{e}-09 \ 1.23502741\text{e}-09]$, $\text{tol}=7.48\text{e}-10$
 Iter 57: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.61233812\text{e}-11]$, $\Delta\text{bounds}=[1.19726214\text{e}-09 \ 9.88027604\text{e}-10]$, $\text{tol}=5.99\text{e}-10$
 Iter 58: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.09483542\text{e}-11]$, $\Delta\text{bounds}=[9.57809713\text{e}-10 \ 7.90322308\text{e}-10]$, $\text{tol}=4.79\text{e}-10$
 Iter 59: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.67564851\text{e}-11]$, $\Delta\text{bounds}=[7.66247770\text{e}-10 \ 6.32274677\text{e}-10]$, $\text{tol}=3.83\text{e}-10$
 Iter 60: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.33739686\text{e}-11]$, $\Delta\text{bounds}=[6.12998216\text{e}-10 \ 5.05880171\text{e}-10]$, $\text{tol}=3.06\text{e}-10$
 Iter 61: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.07510112\text{e}-11]$, $\Delta\text{bounds}=[4.90398573\text{e}-10 \ 4.04693945\text{e}-10]$, $\text{tol}=2.45\text{e}-10$
 Iter 62: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 8.56009708\text{e}-12]$, $\Delta\text{bounds}=[3.92318858\text{e}-10 \ 3.23756855\text{e}-10]$, $\text{tol}=1.96\text{e}-10$
 Iter 63: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.86434243\text{e}-12]$, $\Delta\text{bounds}=[3.13855087\text{e}-10 \ 2.58972788\text{e}-10]$, $\text{tol}=1.57\text{e}-10$
 Iter 64: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.4907745\text{e}-12]$, $\Delta\text{bounds}=[2.51084069\text{e}-10 \ 2.07183770\text{e}-10]$, $\text{tol}=1.26\text{e}-10$
 Iter 65: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.39276393\text{e}-12]$, $\Delta\text{bounds}=[2.00867256\text{e}-10 \ 1.65746084\text{e}-10]$, $\text{tol}=1.00\text{e}-10$
 Iter 66: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.50586227\text{e}-12]$, $\Delta\text{bounds}=[1.60693804\text{e}-10 \ 1.32613587\text{e}-10]$, $\text{tol}=8.03\text{e}-11$
 Iter 67: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.81835666\text{e}-12]$, $\Delta\text{bounds}=[1.28555044\text{e}-10 \ 1.06088083\text{e}-10]$, $\text{tol}=6.43\text{e}-11$
 Iter 68: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.24392727\text{e}-12]$, $\Delta\text{bounds}=[1.02844035\text{e}-10 \ 8.48709436\text{e}-11]$, $\text{tol}=5.14\text{e}-11$
 Iter 69: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.79944948\text{e}-12]$, $\Delta\text{bounds}=[8.22752279\text{e}-11 \ 6.78880840\text{e}-11]$, $\text{tol}=4.11\text{e}-11$
 Iter 70: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.43934864\text{e}-12]$, $\Delta\text{bounds}=[6.58201823\text{e}-11 \ 5.43119438\text{e}-11]$, $\text{tol}=3.29\text{e}-11$
 Iter 71: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.15157883\text{e}-12]$, $\Delta\text{bounds}=[5.26561458\text{e}-11 \ 4.34492997\text{e}-11]$, $\text{tol}=2.63\text{e}-11$
 Iter 72: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.21207555\text{e}-13]$, $\Delta\text{bounds}=[4.21249167\text{e}-11 \ 3.47595841\text{e}-11]$, $\text{tol}=2.11\text{e}-11$
 Iter 73: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.35189687\text{e}-13]$, $\Delta\text{bounds}=[3.36999333\text{e}-11 \ 2.78110313\text{e}-11]$, $\text{tol}=1.68\text{e}-11$
 Iter 74: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.8969496\text{e}-13]$, $\Delta\text{bounds}=[2.69599467\text{e}-11 \ 2.22455387\text{e}-11]$, $\text{tol}=1.35\text{e}-11$

Iter 75: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.71622741\text{e}-13]$, $\Delta\text{bounds}=[2.15679573\text{e}-11 \ 1.77969306\text{e}-11]$, $\text{tol}=1.08\text{e}-11$
 Iter 76: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.77364806\text{e}-13]$, $\Delta\text{bounds}=[1.72543659\text{e}-11 \ 1.42375001\text{e}-11]$, $\text{tol}=8.63\text{e}-12$
 Iter 77: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.0186964\text{e}-13]$, $\Delta\text{bounds}=[1.38034927\text{e}-11 \ 1.13900001\text{e}-11]$, $\text{tol}=6.90\text{e}-12$
 Iter 78: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.41473508\text{e}-13]$, $\Delta\text{bounds}=[1.10427942\text{e}-11 \ 9.11204445\text{e}-12]$, $\text{tol}=5.52\text{e}-12$
 Iter 79: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.92734717\text{e}-13]$, $\Delta\text{bounds}=[8.83423532\text{e}-12 \ 7.29050154\text{e}-12]$, $\text{tol}=4.42\text{e}-12$
 Iter 80: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.54598556\text{e}-13]$, $\Delta\text{bounds}=[7.06738826\text{e}-12 \ 5.83150195\text{e}-12]$, $\text{tol}=3.53\text{e}-12$
 Iter 81: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.23623334\text{e}-13]$, $\Delta\text{bounds}=[5.65391061\text{e}-12 \ 4.66537919\text{e}-12]$, $\text{tol}=2.83\text{e}-12$
 Iter 82: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.89208715\text{e}-14]$, $\Delta\text{bounds}=[4.52312849\text{e}-12 \ 3.73229225\text{e}-12]$, $\text{tol}=2.26\text{e}-12$
 Iter 83: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.91033905\text{e}-14]$, $\Delta\text{bounds}=[3.61850279\text{e}-12 \ 2.98583380\text{e}-12]$, $\text{tol}=1.81\text{e}-12$
 Iter 84: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.33382236\text{e}-14]$, $\Delta\text{bounds}=[2.89480223\text{e}-12 \ 2.38864484\text{e}-12]$, $\text{tol}=1.45\text{e}-12$
 Iter 85: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.05706588\text{e}-14]$, $\Delta\text{bounds}=[2.31584178\text{e}-12 \ 1.91091587\text{e}-12]$, $\text{tol}=1.16\text{e}-12$
 Iter 86: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.06341627\text{e}-14]$, $\Delta\text{bounds}=[1.85267343\text{e}-12 \ 1.52888813\text{e}-12]$, $\text{tol}=9.26\text{e}-13$
 Iter 87: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.23630012\text{e}-14]$, $\Delta\text{bounds}=[1.48213874\text{e}-12 \ 1.22313271\text{e}-12]$, $\text{tol}=7.41\text{e}-13$
 Iter 88: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.58681965\text{e}-14]$, $\Delta\text{bounds}=[1.18571099\text{e}-12 \ 9.78495063\text{e}-13]$, $\text{tol}=5.93\text{e}-13$
 Iter 89: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.08166817\text{e}-14]$, $\Delta\text{bounds}=[9.48568795\text{e}-13 \ 7.82818255\text{e}-13]$, $\text{tol}=4.74\text{e}-13$
 Iter 90: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.65978342\text{e}-14]$, $\Delta\text{bounds}=[7.58855036\text{e}-13 \ 6.26221297\text{e}-13]$, $\text{tol}=3.79\text{e}-13$
 Iter 91: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.3211654\text{e}-14]$, $\Delta\text{bounds}=[6.07084029\text{e}-13 \ 5.00877118\text{e}-13]$, $\text{tol}=3.04\text{e}-13$
 Iter 92: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.0658141\text{e}-14]$, $\Delta\text{bounds}=[4.85667223\text{e}-13 \ 4.00846023\text{e}-13]$, $\text{tol}=2.43\text{e}-13$
 Iter 93: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 8.49320614\text{e}-15]$, $\Delta\text{bounds}=[3.88533778\text{e}-13 \ 3.20687921\text{e}-13]$, $\text{tol}=1.94\text{e}-13$
 Iter 94: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.7168493\text{e}-15]$, $\Delta\text{bounds}=[3.10827023\text{e}-13 \ 2.56517030\text{e}-13]$, $\text{tol}=1.55\text{e}-13$
 Iter 95: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.49560397\text{e}-15]$, $\Delta\text{bounds}=[2.48661618\text{e}-13 \ 2.05280237\text{e}-13]$, $\text{tol}=1.24\text{e}-13$
 Iter 96: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.3298698\text{e}-15]$, $\Delta\text{bounds}=[1.98929295\text{e}-13 \ 1.64257496\text{e}-13]$, $\text{tol}=9.95\text{e}-14$
 Iter 97: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.44169138\text{e}-15]$, $\Delta\text{bounds}=[1.59143436\text{e}-13 \ 1.31228362\text{e}-13]$, $\text{tol}=7.96\text{e}-14$
 Iter 98: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.83106871\text{e}-15]$, $\Delta\text{bounds}=[1.27314749\text{e}-13 \ 1.05138120\text{e}-13]$, $\text{tol}=6.37\text{e}-14$

```

Iter 99: Δorbit=[0.00000000e+00 2.22044605e-15], Δbounds=[1.01851799e-13
8.39883718e-14], tol=5.09e-14
Iter 100: Δorbit=[0.00000000e+00 1.77635684e-15], Δbounds=[8.14814391e-14
6.72795153e-14], tol=4.07e-14
Iter 101: Δorbit=[0.00000000e+00 1.38777878e-15], Δbounds=[6.51851512e-14
5.37903055e-14], tol=3.26e-14
Iter 102: Δorbit=[0.00000000e+00 1.11022302e-15], Δbounds=[5.21481210e-14
4.30766534e-14], tol=2.61e-14
Iter 103: Δorbit=[0.00000000e+00 9.99200722e-16], Δbounds=[4.17184968e-14
3.45279361e-14], tol=2.09e-14
Iter 104: Δorbit=[0.00000000e+00 6.66133815e-16], Δbounds=[3.33747974e-14
2.75890422e-14], tol=1.67e-14
Iter 105: Δorbit=[0.00000000e+00 5.55111512e-16], Δbounds=[2.66998379e-14
2.19269047e-14], tol=1.33e-14
Iter 106: Δorbit=[0.00000000e+00 5.55111512e-16], Δbounds=[2.13598704e-14
1.75970349e-14], tol=1.07e-14
Iter 107: Δorbit=[0.00000000e+00 3.33066907e-16], Δbounds=[1.70878963e-14
1.40998324e-14], tol=8.54e-15
Iter 108: Δorbit=[0.00000000e+00 3.33066907e-16], Δbounds=[1.36703170e-14
1.13242749e-14], tol=6.84e-15
Iter 109: Δorbit=[0.00000000e+00 2.77555756e-16], Δbounds=[1.09362536e-14
9.15933995e-15], tol=5.47e-15
Iter 110: Δorbit=[0.00000000e+00 1.66533454e-16], Δbounds=[8.74900290e-15
7.32747196e-15], tol=4.37e-15
Iter 111: Δorbit=[0.00000000e+00 1.66533454e-16], Δbounds=[6.99920232e-15
5.88418203e-15], tol=3.50e-15
Iter 112: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[5.59936186e-15
4.60742555e-15], tol=2.80e-15
Iter 113: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[4.47948948e-15
3.66373598e-15], tol=2.24e-15
Iter 114: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[3.58359159e-15
2.88657986e-15], tol=1.79e-15
Iter 115: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[2.86687327e-15
2.38697950e-15], tol=1.43e-15
Iter 116: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[2.29349862e-15
1.94289029e-15], tol=1.15e-15
Iter 117: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[1.83479889e-15
1.55431223e-15], tol=9.17e-16
Iter 118: Δorbit=[0.00000000e+00 5.55111512e-17], Δbounds=[1.46783911e-15
1.22124533e-15], tol=7.34e-16
Iter 119: Δorbit=[0. 0.], Δbounds=[1.17427129e-15 1.05471187e-15], tol=5.87e-16
Iter 120: Δorbit=[0. 0.], Δbounds=[9.39417033e-16 7.21644966e-16], tol=4.70e-16
Converged at iteration 120
CPU times: user 220 ms, sys: 5.17 ms, total: 225 ms
Wall time: 225 ms

```

```
[138]: periodic_orbit_saddle_period3_lower, ds.
↳classify_stability(periodic_orbit_saddle_period3_lower, period, parameters=k)
```

```
[138]: (array([0.          , 0.25377828]),
{'classification': 'saddle',
 'eigenvalues': array([5.90789859+0.j, 0.16926492+0.j]),
 'eigenvectors': array([[ 0.84347661+0.j,  0.94680784+0.j],
 [ 0.53716591+0.j, -0.32179949+0.j]])})
```

3.3.2 Upper period 3

For the center

```
[139]: # Define the symmetry line
symmetry_line = lambda v, parameters: 0.5 * np.ones_like(v)
# Define the type of the function, i.e.,  $x = g(y)$ 
axis = 1
# Define the period
period = 3
# Define the range of the initial search
y_range = (0.55, 0.65)
# Define the number of points in the range
num_points = 10000
# Define the initial conditions
points = np.linspace(y_range[0], y_range[1], num_points)
tolerance = 2 / num_points
```

```
[140]: %%time
periodic_orbit_center_period3_upper = ds.find_periodic_orbit(points, period,
↳parameters=k, tolerance=tolerance, symmetry_line=symmetry_line, axis=axis,
↳verbose=True, tolerance_decay_factor=0.7)
```

```
Iter 0: Δorbit=[0.5          0.61430143], Δbounds=[0.0004  0.00037], tol=2.00e-04
Iter 1: Δorbit=[0.00000000e+00 1.60964791e-06], Δbounds=[0.00028  0.00024818],
tol=1.40e-04
Iter 2: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[0.000196  0.00017374],
tol=9.80e-05
Iter 3: Δorbit=[0. 0.], Δbounds=[0.0001372  0.00012161], tol=6.86e-05
Iter 4: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[9.60400000e-05
8.51301066e-05], tol=4.80e-05
Iter 5: Δorbit=[0. 0.], Δbounds=[6.72280000e-05 5.95910657e-05], tol=3.36e-05
Iter 6: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[4.70596000e-05
4.17137468e-05], tol=2.35e-05
Iter 7: Δorbit=[0.00000000e+00 4.1717918e-09], Δbounds=[3.29417200e-05
2.91996227e-05], tol=1.65e-05
Iter 8: Δorbit=[0.00000000e+00 2.92025426e-09], Δbounds=[2.30592040e-05
2.04397359e-05], tol=1.15e-05
Iter 9: Δorbit=[0.00000000e+00 2.04417816e-09], Δbounds=[1.61414428e-05
```

1.43078151e-05], tol=8.07e-06
 Iter 10: $\Delta\text{orbit}=[0.00000000e+00 \ 1.43092482e-09]$, $\Delta\text{bounds}=[1.12990100e-05 \ 1.00154706e-05]$, tol=5.65e-06
 Iter 11: $\Delta\text{orbit}=[0.00000000e+00 \ 5.00823605e-10]$, $\Delta\text{bounds}=[7.90930697e-06 \ 7.01183106e-06]$, tol=3.95e-06
 Iter 12: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[5.53651488e-06 \ 4.90749073e-06]$, tol=2.77e-06
 Iter 13: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[3.87556042e-06 \ 3.43531447e-06]$, tol=1.94e-06
 Iter 14: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[2.71289229e-06 \ 2.40471377e-06]$, tol=1.36e-06
 Iter 15: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[1.89902460e-06 \ 1.68330021e-06]$, tol=9.50e-07
 Iter 16: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[1.32931722e-06 \ 1.17831009e-06]$, tol=6.65e-07
 Iter 17: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[9.30522056e-07 \ 8.24817070e-07]$, tol=4.65e-07
 Iter 18: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[6.51365439e-07 \ 5.77371949e-07]$, tol=3.26e-07
 Iter 19: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[4.55955807e-07 \ 4.04160364e-07]$, tol=2.28e-07
 Iter 20: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[3.19169065e-07 \ 2.82912255e-07]$, tol=1.60e-07
 Iter 21: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[2.23418346e-07 \ 1.98038578e-07]$, tol=1.12e-07
 Iter 22: $\Delta\text{orbit}=[0.00000000e+00 \ 1.98058236e-11]$, $\Delta\text{bounds}=[1.56392842e-07 \ 1.38627005e-07]$, tol=7.82e-08
 Iter 23: $\Delta\text{orbit}=[0.00000000e+00 \ 1.38642431e-11]$, $\Delta\text{bounds}=[1.09474989e-07 \ 9.70389034e-08]$, tol=5.47e-08
 Iter 24: $\Delta\text{orbit}=[0.00000000e+00 \ 9.70490355e-12]$, $\Delta\text{bounds}=[7.66324926e-08 \ 6.79272324e-08]$, tol=3.83e-08
 Iter 25: $\Delta\text{orbit}=[0.00000000e+00 \ 6.79334367e-12]$, $\Delta\text{bounds}=[5.36427447e-08 \ 4.75490626e-08]$, tol=2.68e-08
 Iter 26: $\Delta\text{orbit}=[0.00000000e+00 \ 2.37765363e-12]$, $\Delta\text{bounds}=[3.75499214e-08 \ 3.32890993e-08]$, tol=1.88e-08
 Iter 27: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[2.62849449e-08 \ 2.32986141e-08]$, tol=1.31e-08
 Iter 28: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[1.83994615e-08 \ 1.63093667e-08]$, tol=9.20e-09
 Iter 29: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[1.28796230e-08 \ 1.14165264e-08]$, tol=6.44e-09
 Iter 30: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[9.01573616e-09 \ 7.99157140e-09]$, tol=4.51e-09
 Iter 31: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[6.31101527e-09 \ 5.59409963e-09]$, tol=3.16e-09
 Iter 32: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[4.41771064e-09 \ 3.91586963e-09]$, tol=2.21e-09
 Iter 33: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[3.09239745e-09 \ 2.74110890e-09]$, tol=1.55e-09
 Iter 34: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[2.16467821e-09 \ 1.91877625e-09]$, tol=1.08e-09
 Iter 35: $\Delta\text{orbit}=[0. \ 0.]$, $\Delta\text{bounds}=[1.51527479e-09 \ 1.34314337e-09]$, tol=7.58e-10
 Iter 36: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[1.06069237e-09 \ 9.40200362e-10]$, tol=5.30e-10
 Iter 37: $\Delta\text{orbit}=[0.00000000e+00 \ 1.11022302e-16]$, $\Delta\text{bounds}=[7.42484685e-10$

6.58140431e-10], tol=3.71e-10
 Iter 38: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[5.19739196\text{e-}10 \ 4.60698146\text{e-}10]$, tol=2.60e-10
 Iter 39: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[3.63817421\text{e-}10 \ 3.22488702\text{e-}10]$, tol=1.82e-10
 Iter 40: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[2.54672283\text{e-}10 \ 2.25742092\text{e-}10]$, tol=1.27e-10
 Iter 41: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.78270509\text{e-}10 \ 1.58019375\text{e-}10]$, tol=8.91e-11
 Iter 42: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.24789346\text{e-}10 \ 1.10613518\text{e-}10]$, tol=6.24e-11
 Iter 43: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[8.73525696\text{e-}11 \ 7.74296183\text{e-}11]$, tol=4.37e-11
 Iter 44: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[6.11468098\text{e-}11 \ 5.42006440\text{e-}11]$, tol=3.06e-11
 Iter 45: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[4.28028168\text{e-}11 \ 3.79405396\text{e-}11]$, tol=2.14e-11
 Iter 46: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.99619773\text{e-}11 \ 2.65585332\text{e-}11]$, tol=1.50e-11
 Iter 47: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.09733897\text{e-}11 \ 1.85909066\text{e-}11]$, tol=1.05e-11
 Iter 48: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 2.22044605\text{e-}16]$, $\Delta\text{bounds}=[1.46813672\text{e-}11 \ 1.30135902\text{e-}11]$, tol=7.34e-12
 Iter 49: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.02768904\text{e-}11 \ 9.10937992\text{e-}12]$, tol=5.14e-12
 Iter 50: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[7.19385662\text{e-}12 \ 6.37667696\text{e-}12]$, tol=3.60e-12
 Iter 51: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[5.03574960\text{e-}12 \ 4.46376269\text{e-}12]$, tol=2.52e-12
 Iter 52: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[3.52495810\text{e-}12 \ 3.12461168\text{e-}12]$, tol=1.76e-12
 Iter 53: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 2.22044605\text{e-}16]$, $\Delta\text{bounds}=[2.46752618\text{e-}12 \ 2.18702834\text{e-}12]$, tol=1.23e-12
 Iter 54: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[1.72728498\text{e-}12 \ 1.53110857\text{e-}12]$, tol=8.64e-13
 Iter 55: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[1.20903287\text{e-}12 \ 1.07158726\text{e-}12]$, tol=6.05e-13
 Iter 56: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[8.46378523\text{e-}13 \ 7.50399742\text{e-}13]$, tol=4.23e-13
 Iter 57: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[5.92415006\text{e-}13 \ 5.25024468\text{e-}13]$, tol=2.96e-13
 Iter 58: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[4.14723811\text{e-}13 \ 3.67816888\text{e-}13]$, tol=2.07e-13
 Iter 59: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[2.90267810\text{e-}13 \ 2.57460719\text{e-}13]$, tol=1.45e-13
 Iter 60: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[2.03170814\text{e-}13 \ 1.80189197\text{e-}13]$, tol=1.02e-13
 Iter 61: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 2.22044605\text{e-}16]$, $\Delta\text{bounds}=[1.42275081\text{e-}13 \ 1.26343380\text{e-}13]$, tol=7.11e-14
 Iter 62: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 2.22044605\text{e-}16]$, $\Delta\text{bounds}=[9.95314942\text{e-}14 \ 8.81517082\text{e-}14]$, tol=4.98e-14
 Iter 63: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[6.97220059\text{e-}14 \ 6.18394225\text{e-}14]$, tol=3.49e-14
 Iter 64: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[4.87943019\text{e-}14 \ 4.35207426\text{e-}14]$, tol=2.44e-14
 Iter 65: $\Delta\text{orbit}=[0. 0.]$, $\Delta\text{bounds}=[3.41948692\text{e-}14 \ 3.05311332\text{e-}14]$, tol=1.71e-14
 Iter 66: $\Delta\text{orbit}=[0.00000000\text{e+}00 \ 1.11022302\text{e-}16]$, $\Delta\text{bounds}=[2.39253062\text{e-}14 \ 2.14273044\text{e-}14]$, tol=1.20e-14

```

Iter 67: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.67088565e-14
1.48769885e-14], tol=8.37e-15
Iter 68: Δorbit=[0.00000000e+00 2.22044605e-16], Δbounds=[1.17683641e-14
1.05471187e-14], tol=5.86e-15
Iter 69: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[8.21565038e-15
7.43849426e-15], tol=4.10e-15
Iter 70: Δorbit=[0. 0.], Δbounds=[5.77315973e-15 5.21804822e-15], tol=2.87e-15
Iter 71: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[3.99680289e-15
3.66373598e-15], tol=2.01e-15
Iter 72: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[2.83106871e-15
2.66453526e-15], tol=1.41e-15
Iter 73: Δorbit=[0. 0.], Δbounds=[1.99840144e-15 1.88737914e-15], tol=9.84e-16
Iter 74: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.33226763e-15
1.33226763e-15], tol=6.89e-16
Iter 75: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[9.43689571e-16
8.88178420e-16], tol=4.82e-16
Converged at iteration 75
CPU times: user 150 ms, sys: 5.62 ms, total: 156 ms
Wall time: 156 ms

```

```

[141]: periodic_orbit_center_period3_upper, ds.
↳ classify_stability(periodic_orbit_center_period3_upper, period, parameters=k)

```

```

[141]: (array([0.5          , 0.61430304]),
{'classification': 'elliptic (quasi-periodic)',
 'eigenvalues': array([-0.93105758+0.36487228j, -0.93105758-0.36487228j]),
 'eigenvectors': array([[ -0.31198762-0.06967997j, -0.31198762+0.06967997j],
 [ 0.94752753+0.j          , 0.94752753+0.j          ]])})

```

Now for the saddle

```

[142]: # Define the symmetry line
symmetry_line = lambda v, parameters: 0.0 * np.ones_like(v)
# Define the type of the function, i.e., x = g(y)
axis = 1
# Define the period
period = 3
# Define the range of the initial search
y_range = (0.65, 0.8)
# Define the number of points in the range
num_points = 10000
# Define the initial conditions
points = np.linspace(y_range[0], y_range[1], num_points)
tolerance = 2 / num_points

```

```

[143]: %%time

```

```
periodic_orbit_saddle_period3_upper = ds.find_periodic_orbit(points, period,
↳parameters=k, tolerance=tolerance, symmetry_line=symmetry_line, axis=axis,
↳verbose=True, tolerance_decay_factor=0.7)
```

```
Iter 0: Δorbit=[0.          0.74625713], Δbounds=[0.0004   0.000355],
tol=2.00e-04
Iter 1: Δorbit=[0.00000000e+00 1.09172033e-05], Δbounds=[0.00028   0.00023108],
tol=1.40e-04
Iter 2: Δorbit=[0.00000000e+00 7.34897484e-06], Δbounds=[0.000196   0.00016173],
tol=9.80e-05
Iter 3: Δorbit=[0.00000000e+00 5.14346138e-06], Δbounds=[0.0001372   0.00011321],
tol=6.86e-05
Iter 4: Δorbit=[0.00000000e+00 3.60054473e-06], Δbounds=[9.60400000e-05
7.92487804e-05], tol=4.80e-05
Iter 5: Δorbit=[0.00000000e+00 2.52036325e-06], Δbounds=[6.72280000e-05
5.54742305e-05], tol=3.36e-05
Iter 6: Δorbit=[0.00000000e+00 1.76425696e-06], Δbounds=[4.70596000e-05
3.88319489e-05], tol=2.35e-05
Iter 7: Δorbit=[0.00000000e+00 1.23497947e-06], Δbounds=[3.2941720e-05
2.7182366e-05], tol=1.65e-05
Iter 8: Δorbit=[0.00000000e+00 8.64485689e-07], Δbounds=[2.3059204e-05
1.9027656e-05], tol=1.15e-05
Iter 9: Δorbit=[0.00000000e+00 6.04188496e-07], Δbounds=[1.61414428e-05
1.33212622e-05], tol=8.07e-06
Iter 10: Δorbit=[0.00000000e+00 4.23658503e-07], Δbounds=[1.12990100e-05
9.32326921e-06], tol=5.65e-06
Iter 11: Δorbit=[0.00000000e+00 2.96509612e-07], Δbounds=[7.90930697e-06
6.52652787e-06], tol=3.95e-06
Iter 12: Δorbit=[0.00000000e+00 2.07564343e-07], Δbounds=[5.53651488e-06
4.56853400e-06], tol=2.77e-06
Iter 13: Δorbit=[0.00000000e+00 1.45065461e-07], Δbounds=[3.87556042e-06
3.19843597e-06], tol=1.94e-06
Iter 14: Δorbit=[0.00000000e+00 1.01720436e-07], Δbounds=[2.71289229e-06
2.23851680e-06], tol=1.36e-06
Iter 15: Δorbit=[0.00000000e+00 7.11919534e-08], Δbounds=[1.89902460e-06
1.56701936e-06], tol=9.50e-07
Iter 16: Δorbit=[0.00000000e+00 4.98361994e-08], Δbounds=[1.32931722e-06
1.09690501e-06], tol=6.65e-07
Iter 17: Δorbit=[0.00000000e+00 3.48850676e-08], Δbounds=[9.30522056e-07
7.67834774e-07], tol=4.65e-07
Iter 18: Δorbit=[0.00000000e+00 2.43811923e-08], Δbounds=[6.51365439e-07
5.37560945e-07], tol=3.26e-07
Iter 19: Δorbit=[0.00000000e+00 1.70961477e-08], Δbounds=[4.55955807e-07
3.76227546e-07], tol=2.28e-07
Iter 20: Δorbit=[0.00000000e+00 1.19652325e-08], Δbounds=[3.19169065e-07
2.63368940e-07], tol=1.60e-07
Iter 21: Δorbit=[0.00000000e+00 8.37597003e-09], Δbounds=[2.23418346e-07
1.84356826e-07], tol=1.12e-07
```

Iter 22: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.86313331\text{e}-09]$, $\Delta\text{bounds}=[1.56392842\text{e}-07 \ 1.29049991\text{e}-07]$, $\text{tol}=7.82\text{e}-08$
 Iter 23: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.10420009\text{e}-09]$, $\Delta\text{bounds}=[1.09474989\text{e}-07 \ 9.03349617\text{e}-08]$, $\text{tol}=5.47\text{e}-08$
 Iter 24: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.86842194\text{e}-09]$, $\Delta\text{bounds}=[7.66324926\text{e}-08 \ 6.32435123\text{e}-08]$, $\text{tol}=3.83\text{e}-08$
 Iter 25: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.01134476\text{e}-09]$, $\Delta\text{bounds}=[5.36427448\text{e}-08 \ 4.42627939\text{e}-08]$, $\text{tol}=2.68\text{e}-08$
 Iter 26: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.40769763\text{e}-09]$, $\Delta\text{bounds}=[3.75499214\text{e}-08 \ 3.09850925\text{e}-08]$, $\text{tol}=1.88\text{e}-08$
 Iter 27: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.83875204\text{e}-10]$, $\Delta\text{bounds}=[2.62849449\text{e}-08 \ 2.16924949\text{e}-08]$, $\text{tol}=1.31\text{e}-08$
 Iter 28: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.90974944\text{e}-10]$, $\Delta\text{bounds}=[1.83994615\text{e}-08 \ 1.51843123\text{e}-08]$, $\text{tol}=9.20\text{e}-09$
 Iter 29: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.82150209\text{e}-10]$, $\Delta\text{bounds}=[1.28796230\text{e}-08 \ 1.06290828\text{e}-08]$, $\text{tol}=6.44\text{e}-09$
 Iter 30: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.37507133\text{e}-10]$, $\Delta\text{bounds}=[9.01573612\text{e}-09 \ 7.44034845\text{e}-09]$, $\text{tol}=4.51\text{e}-09$
 Iter 31: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.36626718\text{e}-10]$, $\Delta\text{bounds}=[6.31101528\text{e}-09 \ 5.20750121\text{e}-09]$, $\text{tol}=3.16\text{e}-09$
 Iter 32: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.65614966\text{e}-10]$, $\Delta\text{bounds}=[4.41771070\text{e}-09 \ 3.64536101\text{e}-09]$, $\text{tol}=2.21\text{e}-09$
 Iter 33: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.15934151\text{e}-10]$, $\Delta\text{bounds}=[3.09239749\text{e}-09 \ 2.55173638\text{e}-09]$, $\text{tol}=1.55\text{e}-09$
 Iter 34: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 8.11531953\text{e}-11]$, $\Delta\text{bounds}=[2.16467824\text{e}-09 \ 1.78621784\text{e}-09]$, $\text{tol}=1.08\text{e}-09$
 Iter 35: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.68074476\text{e}-11]$, $\Delta\text{bounds}=[1.51527477\text{e}-09 \ 1.25035216\text{e}-09]$, $\text{tol}=7.58\text{e}-10$
 Iter 36: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.97026856\text{e}-11]$, $\Delta\text{bounds}=[1.06069234\text{e}-09 \ 8.75371664\text{e}-10]$, $\text{tol}=5.30\text{e}-10$
 Iter 37: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.78397305\text{e}-11]$, $\Delta\text{bounds}=[7.42484637\text{e}-10 \ 6.12654150\text{e}-10]$, $\text{tol}=3.71\text{e}-10$
 Iter 38: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.9484192\text{e}-11]$, $\Delta\text{bounds}=[5.19739246\text{e}-10 \ 4.28873492\text{e}-10]$, $\text{tol}=2.60\text{e}-10$
 Iter 39: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.36181066\text{e}-11]$, $\Delta\text{bounds}=[3.63817472\text{e}-10 \ 3.00251934\text{e}-10]$, $\text{tol}=1.82\text{e}-10$
 Iter 40: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.56412727\text{e}-12]$, $\Delta\text{bounds}=[2.54672230\text{e}-10 \ 2.10170548\text{e}-10]$, $\text{tol}=1.27\text{e}-10$
 Iter 41: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.6735506\text{e}-12]$, $\Delta\text{bounds}=[1.78270561\text{e}-10 \ 1.47120094\text{e}-10]$, $\text{tol}=8.91\text{e}-11$
 Iter 42: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.67148542\text{e}-12]$, $\Delta\text{bounds}=[1.24789393\text{e}-10 \ 1.02983955\text{e}-10]$, $\text{tol}=6.24\text{e}-11$
 Iter 43: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.28015393\text{e}-12]$, $\Delta\text{bounds}=[8.73525750\text{e}-11 \ 7.20888904\text{e}-11]$, $\text{tol}=4.37\text{e}-11$
 Iter 44: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.28916885\text{e}-12]$, $\Delta\text{bounds}=[6.11468025\text{e}-11 \ 5.04621900\text{e}-11]$, $\text{tol}=3.06\text{e}-11$
 Iter 45: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.60238489\text{e}-12]$, $\Delta\text{bounds}=[4.28027618\text{e}-11 \ 3.53236329\text{e}-11]$, $\text{tol}=2.14\text{e}-11$

Iter 46: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.12532206\text{e}-12]$, $\Delta\text{bounds}=[2.99619332\text{e}-11 \ 2.47265541\text{e}-11]$, $\text{tol}=1.50\text{e}-11$
 Iter 47: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.85038701\text{e}-13]$, $\Delta\text{bounds}=[2.09733533\text{e}-11 \ 1.73085990\text{e}-11]$, $\text{tol}=1.05\text{e}-11$
 Iter 48: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.50559598\text{e}-13]$, $\Delta\text{bounds}=[1.46813473\text{e}-11 \ 1.21143096\text{e}-11]$, $\text{tol}=7.34\text{e}-12$
 Iter 49: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.85136367\text{e}-13]$, $\Delta\text{bounds}=[1.02769431\text{e}-11 \ 8.48010551\text{e}-12]$, $\text{tol}=5.14\text{e}-12$
 Iter 50: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.69229083\text{e}-13]$, $\Delta\text{bounds}=[7.19386017\text{e}-12 \ 5.93691762\text{e}-12]$, $\text{tol}=3.60\text{e}-12$
 Iter 51: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.88959959\text{e}-13]$, $\Delta\text{bounds}=[5.03570212\text{e}-12 \ 4.15523171\text{e}-12]$, $\text{tol}=2.52\text{e}-12$
 Iter 52: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.32005518\text{e}-13]$, $\Delta\text{bounds}=[3.52499148\text{e}-12 \ 2.90867330\text{e}-12]$, $\text{tol}=1.76\text{e}-12$
 Iter 53: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 9.22595333\text{e}-14]$, $\Delta\text{bounds}=[2.46749404\text{e}-12 \ 2.03648209\text{e}-12]$, $\text{tol}=1.23\text{e}-12$
 Iter 54: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 6.49480469\text{e}-14]$, $\Delta\text{bounds}=[1.72724583\text{e}-12 \ 1.42530432\text{e}-12]$, $\text{tol}=8.64\text{e}-13$
 Iter 55: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 4.52970994\text{e}-14]$, $\Delta\text{bounds}=[1.20907208\text{e}-12 \ 9.97535388\text{e}-13]$, $\text{tol}=6.05\text{e}-13$
 Iter 56: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.18634008\text{e}-14]$, $\Delta\text{bounds}=[8.46350455\text{e}-13 \ 6.98441305\text{e}-13]$, $\text{tol}=4.23\text{e}-13$
 Iter 57: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.20934382\text{e}-14]$, $\Delta\text{bounds}=[5.92445319\text{e}-13 \ 4.88831198\text{e}-13]$, $\text{tol}=2.96\text{e}-13$
 Iter 58: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.55431223\text{e}-14]$, $\Delta\text{bounds}=[4.14711723\text{e}-13 \ 3.42392781\text{e}-13]$, $\text{tol}=2.07\text{e}-13$
 Iter 59: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.09912079\text{e}-14]$, $\Delta\text{bounds}=[2.90298206\text{e}-13 \ 2.39475106\text{e}-13]$, $\text{tol}=1.45\text{e}-13$
 Iter 60: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.54951657\text{e}-15]$, $\Delta\text{bounds}=[2.03208744\text{e}-13 \ 1.67754699\text{e}-13]$, $\text{tol}=1.02\text{e}-13$
 Iter 61: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 5.32907052\text{e}-15]$, $\Delta\text{bounds}=[1.42246121\text{e}-13 \ 1.17572618\text{e}-13]$, $\text{tol}=7.11\text{e}-14$
 Iter 62: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.55271368\text{e}-15]$, $\Delta\text{bounds}=[9.95722847\text{e}-14 \ 8.20454815\text{e}-14]$, $\text{tol}=4.98\text{e}-14$
 Iter 63: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 2.77555756\text{e}-15]$, $\Delta\text{bounds}=[6.97005993\text{e}-14 \ 5.76205750\text{e}-14]$, $\text{tol}=3.49\text{e}-14$
 Iter 64: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.77635684\text{e}-15]$, $\Delta\text{bounds}=[4.87904195\text{e}-14 \ 4.04121181\text{e}-14]$, $\text{tol}=2.44\text{e}-14$
 Iter 65: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 1.22124533\text{e}-15]$, $\Delta\text{bounds}=[3.41532937\text{e}-14 \ 2.81996648\text{e}-14]$, $\text{tol}=1.71\text{e}-14$
 Iter 66: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.77156117\text{e}-16]$, $\Delta\text{bounds}=[2.39073056\text{e}-14 \ 1.97619698\text{e}-14]$, $\text{tol}=1.20\text{e}-14$
 Iter 67: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 7.77156117\text{e}-16]$, $\Delta\text{bounds}=[1.67351139\text{e}-14 \ 1.37667655\text{e}-14]$, $\text{tol}=8.37\text{e}-15$
 Iter 68: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.33066907\text{e}-16]$, $\Delta\text{bounds}=[1.17145797\text{e}-14 \ 9.76996262\text{e}-15]$, $\text{tol}=5.86\text{e}-15$
 Iter 69: $\Delta\text{orbit}=[0.00000000\text{e}+00 \ 3.33066907\text{e}-16]$, $\Delta\text{bounds}=[8.20020581\text{e}-15 \ 6.77236045\text{e}-15]$, $\text{tol}=4.10\text{e}-15$

```

Iter 70: Δorbit=[0.00000000e+00 3.33066907e-16], Δbounds=[5.74014406e-15
4.88498131e-15], tol=2.87e-15
Iter 71: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[4.01810085e-15
3.44169138e-15], tol=2.01e-15
Iter 72: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[2.81267059e-15
2.44249065e-15], tol=1.41e-15
Iter 73: Δorbit=[0. 0.], Δbounds=[1.96886941e-15 1.66533454e-15], tol=9.84e-16
Iter 74: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[1.37820859e-15
1.22124533e-15], tol=6.89e-16
Iter 75: Δorbit=[0.00000000e+00 1.11022302e-16], Δbounds=[9.64746013e-16
7.77156117e-16], tol=4.82e-16
Converged at iteration 75
CPU times: user 147 ms, sys: 2.86 ms, total: 150 ms
Wall time: 151 ms

```

```

[144]: periodic_orbit_saddle_period3_upper, ds.
       ↪classify_stability(periodic_orbit_saddle_period3_upper, period, parameters=k)

```

```

[144]: (array([0.          , 0.74622172]),
       {'classification': 'saddle',
        'eigenvalues': array([5.90789859+0.j, 0.16926492+0.j]),
        'eigenvectors': array([[ 0.84347661+0.j,  0.94680784+0.j],
                                [ 0.53716591+0.j, -0.32179949+0.j]])})

```

3.3.3 Now the manifolds

```

[145]: %%time
       n_points = 50000
       iter_time = 18
       wu_period3_lower = ds.manifold(periodic_orbit_saddle_period3_lower, period,
       ↪parameters=k, n_points=n_points, iter_time=iter_time, stability="unstable")
       ws_period3_lower = ds.manifold(periodic_orbit_saddle_period3_lower, period,
       ↪parameters=k, n_points=n_points, iter_time=iter_time, stability="stable")

```

```

CPU times: user 238 ms, sys: 14.4 ms, total: 253 ms
Wall time: 46.7 ms

```

```

[146]: %%time
       n_points = 50000
       iter_time = 18
       wu_period3_upper = ds.manifold(periodic_orbit_saddle_period3_upper, period,
       ↪parameters=k, n_points=n_points, iter_time=iter_time, stability="unstable")
       ws_period3_upper = ds.manifold(periodic_orbit_saddle_period3_upper, period,
       ↪parameters=k, n_points=n_points, iter_time=iter_time, stability="stable")

```

```

CPU times: user 254 ms, sys: 11.9 ms, total: 266 ms
Wall time: 39.3 ms

```

3.4 Final plot

```
[149]: num_ic = 100
total_time = 30000
np.random.seed(11331313)
u = np.random.rand(num_ic, 2)
k = 1.5
trajectories = ds.trajectory(u, total_time, parameters=k)

[152]: ps = PlotStyler(fontsize=12, legend_fontsize=7, axes_linewidth=1.1)
ps.apply_style()
fig, ax = plt.subplots()
ps.set_tick_padding(ax, pad_x=5)
plt.plot(trajectories[:, 0].T, trajectories[:, 1], "ko", markersize=0.1,
↪markeredgewidth=0.0)
ms = 0.75
pms = 4

plt.plot(0.5, 0, "s", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon", label=r"$\mathbf{E}_1$")
plt.plot(0.5, 1, "s", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon")
plt.plot(0, 0, "o", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon", label=r"$\mathbf{H}_1$")
plt.plot(0, 1, "o", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon")
plt.plot(1, 0, "o", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon")
plt.plot(1, 1, "o", markersize=pms, markeredgewidth="k", clip_on=False,
↪zorder=3, color="maroon")

plt.plot(0, 0, "r", label="$W^s_1$")
plt.plot(0, 0, label="$W^u_1$", color="maroon")

plt.plot(ws_period1[0][:, 0], ws_period1[0][:, 1], "o", markersize=ms,
↪markeredgewidth=0.0, color="red") # along v
plt.plot(ws_period1[1][:, 0], ws_period1[1][:, 1], "o", markersize=ms,
↪markeredgewidth=0.0, color="red") # along -v
plt.plot(wu_period1[0][:, 0], wu_period1[0][:, 1], "o", markersize=ms,
↪markeredgewidth=0.0, color="maroon") # along v
plt.plot(wu_period1[1][:, 0], wu_period1[1][:, 1], "o", markersize=ms,
↪markeredgewidth=0.0, color="maroon") # along -v

ts = ds.trajectory(periodic_orbit_center_p2, 2, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "bs", markersize=pms, markeredgewidth="k",
↪clip_on=False, zorder=3, label=r"$\mathbf{E}_2$")
```

```

plt.plot(1, 0.5 , "bs", markersize=pms, markeredgecolor="k", clip_on=False,
↳zorder=3)
ts = ds.trajectory(periodic_orbit_saddle_p2, 2, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "bo", markersize=pms, markeredgecolor="k",
↳clip_on=False, zorder=3, label=r"$\mathbf{H}_2$")

plt.plot(ws_period2[0][:, 0], ws_period2[0][:, 1], "o", markersize=ms,
↳markeredgewidth=0.0, color="deepskyblue") # along v
plt.plot(ws_period2[1][:, 0], ws_period2[1][:, 1], "o", markersize=ms,
↳markeredgewidth=0.0, color="deepskyblue") # along -v
plt.plot(wu_period2[0][:, 0], wu_period2[0][:, 1], "bo", markersize=ms,
↳markeredgewidth=0.0) # along v
plt.plot(wu_period2[1][:, 0], wu_period2[1][:, 1], "bo", markersize=ms,
↳markeredgewidth=0.0) # along -v

plt.plot(0, 0, label="$W^s_2$", color="deepskyblue")
plt.plot(0, 0, label="$W^u_2$", color="blue")

ts = ds.trajectory(periodic_orbit_center_period3_lower, 3, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "gs", markersize=pms, markeredgecolor="k",
↳clip_on=False, zorder=3, label=r"$\mathbf{E}_3$")
plt.plot(1, 0.5 , "bs", markersize=pms, markeredgecolor="k", clip_on=False,
↳zorder=3)
ts = ds.trajectory(periodic_orbit_saddle_period3_lower, 3, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "go", markersize=pms, markeredgecolor="k",
↳clip_on=False, zorder=3, label=r"$\mathbf{H}_3$")

plt.plot(ws_period3_lower[0][:, 0], ws_period3_lower[0][:, 1], "o",
↳markersize=ms, markeredgewidth=0.0, color="springgreen") # along v
plt.plot(ws_period3_lower[1][:, 0], ws_period3_lower[1][:, 1], "o",
↳markersize=ms, markeredgewidth=0.0, color="springgreen") # along -v
plt.plot(wu_period3_lower[0][:, 0], wu_period3_lower[0][:, 1], "go",
↳markersize=ms, markeredgewidth=0.0) # along v
plt.plot(wu_period3_lower[1][:, 0], wu_period3_lower[1][:, 1], "go",
↳markersize=ms, markeredgewidth=0.0) # along -v

plt.plot(0, 0, label="$W^s_3$", color="springgreen")
plt.plot(0, 0, label="$W^u_3$", color="green")

ts = ds.trajectory(periodic_orbit_center_period3_upper, 3, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "s", markersize=pms, markeredgecolor="k",
↳clip_on=False, zorder=3, color="darkviolet", label=r"$\mathbf{E}_3$")
ts = ds.trajectory(periodic_orbit_saddle_period3_upper, 3, parameters=k)
plt.plot(ts[:, 0], ts[:, 1], "o", markersize=pms, markeredgecolor="k",
↳clip_on=False, zorder=3, color="darkviolet", label=r"$\mathbf{H}_3$")

```

```

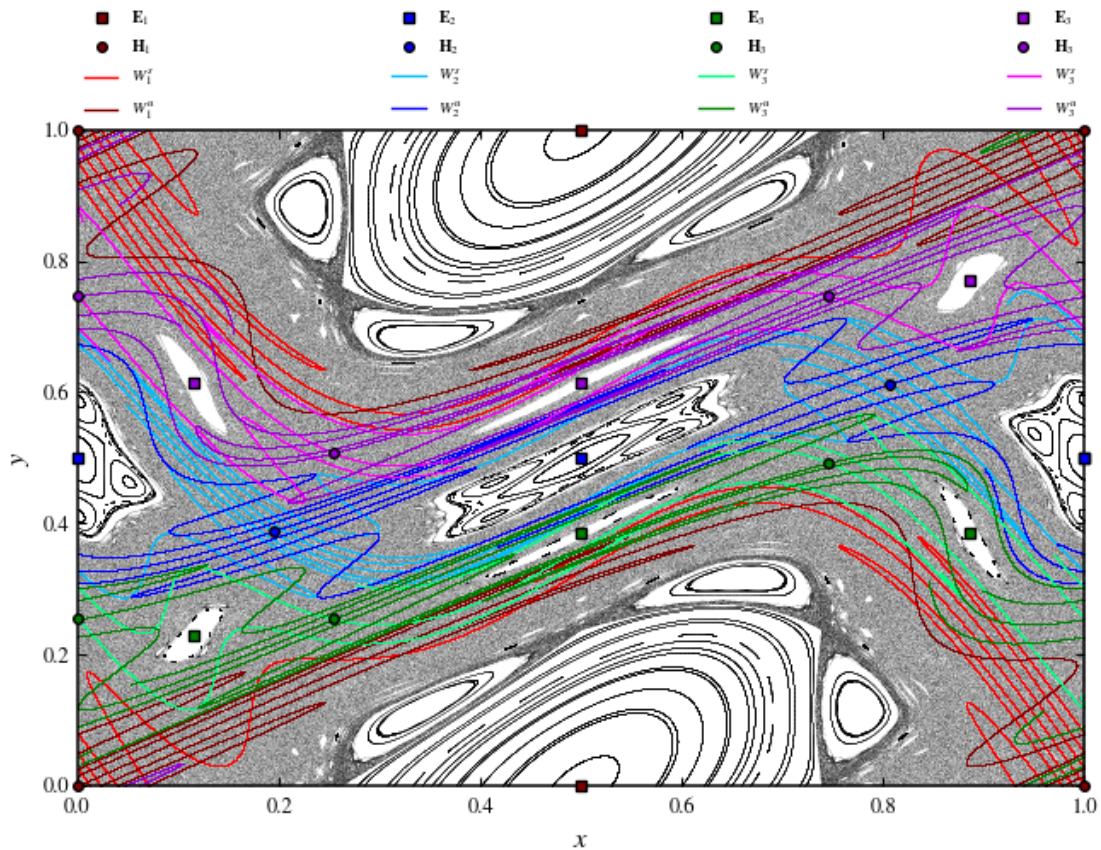
plt.plot(ws_period3_upper[0][:, 0], ws_period3_upper[0][:, 1], "o",
        ↪markersize=ms, markeredgewidth=0.0, color="fuchsia") # along v
plt.plot(ws_period3_upper[1][:, 0], ws_period3_upper[1][:, 1], "o",
        ↪markersize=ms, markeredgewidth=0.0, color="fuchsia") # along -v
plt.plot(wu_period3_upper[0][:, 0], wu_period3_upper[0][:, 1], "o",
        ↪markersize=ms, markeredgewidth=0.0, color="darkviolet") # along v
plt.plot(wu_period3_upper[1][:, 0], wu_period3_upper[1][:, 1], "o",
        ↪markersize=ms, markeredgewidth=0.0, color="darkviolet") # along -v

plt.plot(0, 0, label="$W^s_3$", color="fuchsia")
plt.plot(0, 0, label="$W^u_3$", color="darkviolet")

# plt.legend(loc="upper center", fontsize=8, frameon=False, handlelength=1.5,
        ↪handletextpad=0.5, borderpad=0.5, bbox_to_anchor=(0.5, 1.5), ncol=4)
plt.legend(bbox_to_anchor=(0, 1.0, 1, 0.2), loc="lower left",
        mode="expand", borderaxespad=0, ncol=4, frameon=False,
        ↪fancybox=False)
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.xlabel(r"$x$")
plt.ylabel(r"$y$")
plt.subplots_adjust(left=0.065, bottom=0.08, right=0.987, top=0.88, wspace=0.2,
        ↪hspace=0.2)
plt.savefig("fig11.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



4 Escape

4.1 Survival probability

```
[153]: ds = dds(model="leonel map")
```

```
[154]: eps, gamma = 1e-3, 1
        parameters = [eps, gamma]
```

```
[155]: total_time = 5000000
        u = [np.pi, 1e-15]
```

```
[158]: %%time
        trajectory = ds.trajectory(u, total_time, parameters=parameters)
```

CPU times: user 373 ms, sys: 5.25 ms, total: 378 ms

Wall time: 378 ms

```
[159]: ds.info["parameters"]
```

```
[159]: ['eps', 'gamma']
```

```
[116]: max_time = 1000000
num_ic = 1000000
np.random.seed(13)
x_range = (0, 2 * np.pi, num_ic)
y_range = (-1e-14, 1e-14, num_ic)
x = np.random.uniform(*x_range)
y = np.random.uniform(*y_range)
y_esc = np.logspace(np.log10(1e-3), np.log10(0.025), 25)
x_esc = (0, 2 * np.pi)
sp, times = [], []
```

```
[117]: %%time
for i in range(y_esc.shape[0]):
    exit = np.array([x_esc[0], x_esc[1]], [-y_esc[i], y_esc[i]])
    escape = np.array(Parallel(n_jobs=-1)(delayed(ds.escape_analysis)([x[j],
    ↪y[j]], max_time, exit, parameters=parameters, escape="exiting") for j in
    ↪range(num_ic)))
    time, survival_probability = ds.survival_probability(escape[:, 1], escape[:,
    ↪1].max())
    times.append(time)
    sp.append(survival_probability)
```

CPU times: user 5min 51s, sys: 5.32 s, total: 5min 56s

Wall time: 7min 54s

```
[121]: colors = sns.color_palette("icefire", len(y_esc))
cmap = ListedColormap(colors)
Y_esc = np.array(y_esc)
norm = mpl.colors.Normalize(vmin=min(Y_esc), vmax=max(Y_esc))
sm = mpl.cm.ScalarMappable(cmap=cmap, norm=norm)
```

```
[191]: fontsize=17
ps = PlotStyler(fontsize=fontsize, ticks_on_all_sides=False, markersize=0.1,
    ↪markeredgewidth=0)
ps.apply_style()

fig, ax = plt.subplots(1, 2, figsize=(10, 3))
[ps.set_tick_padding(ax[i], pad_x=5) for i in range(ax.shape[0])]
plt.subplots_adjust(left=0.075, bottom=0.16, right=1.065, top=0.975)

ax[0].plot(trajjectory[:, 0], trajectory[:, 1], "ko")
ax[0].set_xlim(0, 2 * np.pi)
ax[0].set_xlabel("$x$")
ax[0].set_ylabel("$y$")
ax[0].set_xticks([0, np.pi/2, np.pi, 3 * np.pi / 2, 2 * np.pi], [r"$0$", r"$\pi/$
    ↪2$", r"$\pi$", r"$3\pi/2$", r"$2\pi$"])
```

```

ax[0].set_yticks([-0.03, -0.02, -0.01, 0, 0.01, 0.02, 0.03])

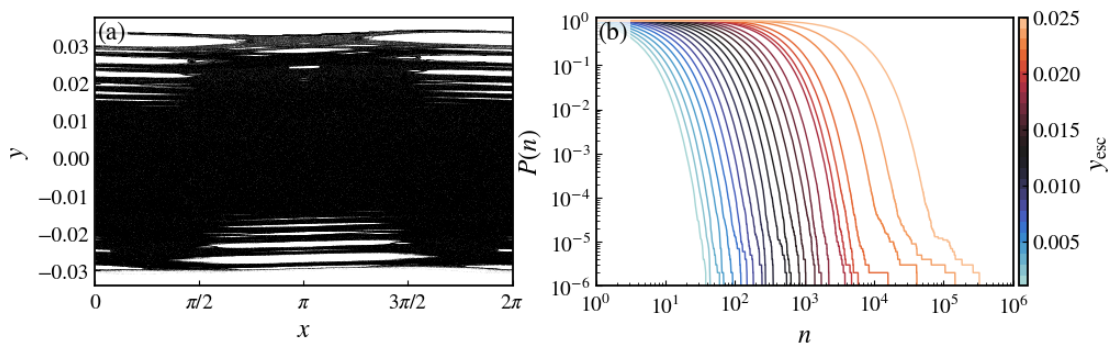
for i in range(y_esc.shape[0]):
    ax[1].plot(times[i], sp[i], color=colors[i])
ax[1].set_xscale("log")
ax[1].set_yscale("log")
ax[1].set_ylim(1 / num_ic, 1.2e0)
ax[1].set_xlim(1e0, 1e6)
ax[1].set_xlabel("$n$")
ax[1].set_ylabel("$P(n)$")
fig.colorbar(sm, ax=ax, pad=0.005, aspect=30, label=r"$y_{\mathrm{esc}}$")

xbox = 0.0066
ybox = 0.923
bbox = {"facecolor": "w", "pad": 1, "alpha": 0.75, "linewidth": 0.0}
[ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", transform=ax[i].transAxes,
  ↳ bbox=bbox) for i in range(ax.shape[0])]

plt.savefig("fig12.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



4.2 Escape basins

```
[3]: from numba import njit
```

```

[4]: @njit
def weiss_map(u, parameters):
    k = parameters[0]
    x, y = u
    y_new = y - k * np.sin(x)
    x_new = (x + k * (y_new ** 2 - 1) + np.pi) % (2 * np.pi) - np.pi

```

```
return np.array([x_new, y_new])
```

```
[5]: ds = dds(mapping=weiss_map, system_dimension=2, number_of_parameters=1)
```

```
[6]: import numpy as np

centers = np.array([[0.0, -1.1],
                    [np.pi - 0.1, 1.0]], dtype=np.float64)
size_exit = 0.2
```

```
[7]: ks = [0.5, 0.55, 0.60, 0.70]
total_time = 10000
```

```
[26]: grid_size = 1000
x_range = (-np.pi, np.pi, grid_size)
y_range = (-np.pi, np.pi, grid_size)
X = np.linspace(*x_range)
Y = np.linspace(*y_range)
```

```
[27]: from joblib import Parallel, delayed
import itertools
```

```
[28]: escapes = np.zeros((len(ks), grid_size, grid_size, 2))
```

```
[43]: %%time
for i, k in enumerate(ks):

    escape = Parallel(n_jobs=-1)(
        delayed(ds.escape_analysis)([x, y], total_time, centers, parameters=k,
hole_size=size_exit)
        for x, y in itertools.product(X, Y)
    )
    escape = np.array(escape).reshape(grid_size, grid_size, 2)

    escapes[i, :, :, :] = escape
```

CPU times: user 55.1 s, sys: 1.37 s, total: 56.4 s

Wall time: 3min 39s

```
[11]: from matplotlib.colors import ListedColormap, BoundaryNorm
import matplotlib as mpl
```

```
[45]: colors = ["green", "gold", "blue"]
cmap = ListedColormap(colors)
bounds = [-1.5, -0.5, 0.5, 1.5]
norm = BoundaryNorm(boundaries=bounds, ncolors=len(colors))
```

```

[46]: ps = PlotStyler(fontsize=18)
ps.apply_style()
fig, ax = plt.subplots(2, 4, sharex=True, sharey=True, figsize=(10, 5))
# plt.tight_layout(pad=0)
plt.subplots_adjust(left=0.055, bottom=0.095, top=0.995, right=0.945, hspace=0.
    ↪08, wspace=0.15)

x_grid, y_grid = np.meshgrid(X, Y, indexing='ij')
for i, k in enumerate(ks):
    hm1 = ax[0, i].pcolormesh(x_grid, y_grid, escapes[i, :, :, 0], cmap=cmap,
    ↪norm=norm)
    hm2 = ax[1, i].pcolormesh(x_grid, y_grid, escapes[i, :, :, 1],
    ↪cmap="nipy_spectral", norm=matplotlib.colors.LogNorm(vmin=1e0, vmax=total_time))
    ax[1, i].set_xlabel(r"$x$")

ax[0, 0].set_ylabel(r"$y$")
ax[1, 0].set_ylabel(r"$y$")
ax[0, 0].set_xticks([-np.pi, 0, np.pi])
ax[0, 0].set_xticklabels([r"$-\pi$", r"$0$", r"$\pi$"])
ax[0, 0].set_yticks([-np.pi, 0, np.pi])
ax[0, 0].set_yticklabels([r"$-\pi$", r"$0$", r"$\pi$"])
ax[0, 0].set_xlim(-np.pi, np.pi)
ax[0, 0].set_ylim(-np.pi, np.pi)
cbar1 = fig.colorbar(hm1, ax=ax[0, :], aspect=20, pad=0.005, fraction=0.02)
cbar1.set_label(r"Escape basin")
cbar1.set_ticks([-1, 0, 1])
cbar1.set_ticklabels([r"$\mathcal{B}_{-\infty}$", r"$\mathcal{B}_0$",
    ↪r"$\mathcal{B}_1$"])

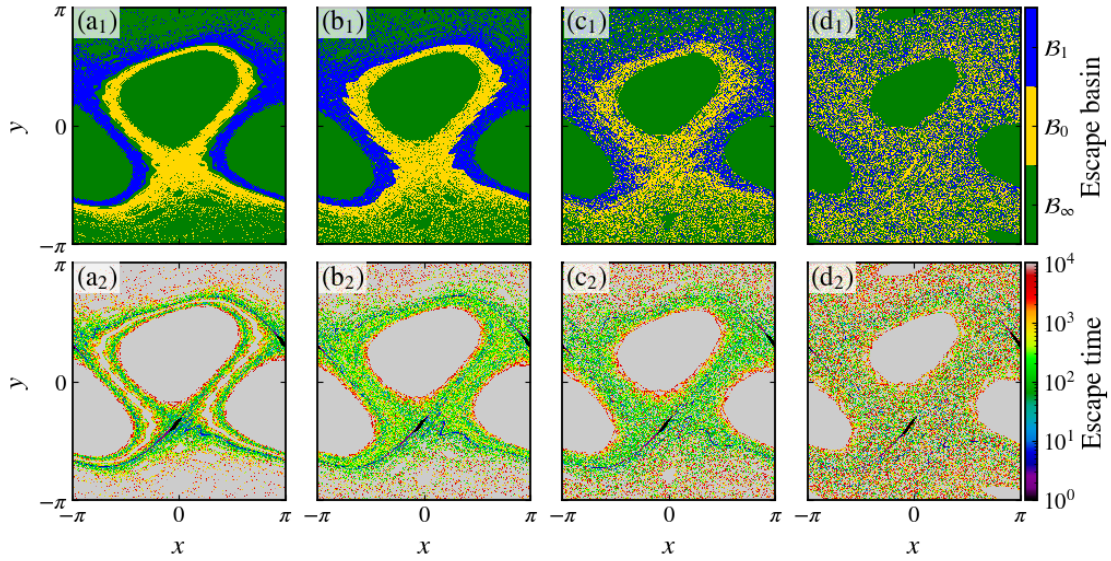
cbar2 = fig.colorbar(hm2, ax=ax[1, :], aspect=20, pad=0.005, fraction=0.02)
cbar2.set_label(r"Escape time")

xbox = 0.0143
ybox = 0.908
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}
for i in range(4):
    ax[0, i].text(xbox, ybox, f"({ascii_lowercase[i]}$_1$)", transform=ax[0, i].
    ↪transAxes, bbox=bbox)
    ax[1, i].text(xbox, ybox, f"({ascii_lowercase[i]}$_2$)", transform=ax[1, i].
    ↪transAxes, bbox=bbox)

plt.savefig("fig13.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



```
[29]: ks = np.linspace(0.2, 1.0, 100)
      escape_basins = np.zeros((len(ks), grid_size, grid_size))
      total_time = 10000
      Sb = []
      Sbb = []
      D = []
```

```
[30]: from dynamicalsys import BasinMetrics
```

```
[31]: x_grid, y_grid = np.meshgrid(X, Y, indexing="ij")
```

```
[32]: %%time
      for i, k in enumerate(ks):

          escape = Parallel(n_jobs=-1)(
              delayed(ds.escape_analysis)([x, y], total_time, centers, parameters=k,
              ↪hole_size=size_exit)
              for x, y in itertools.product(X, Y)
          )
          escape = np.array(escape).reshape(grid_size, grid_size, 2)
          escape_basins[i] = escape[:, :, 0]

          bm = BasinMetrics(escape[:, :, 0])
          basin_entropy = bm.basin_entropy(5, log_base=2)
          Sb.append(basin_entropy[0])
          Sbb.append(basin_entropy[1])
          eps, f = bm.uncertainty_fraction(x_grid, y_grid, )
          alpha, _ = np.polyfit(np.log(eps), np.log(f), 1)
```

```
D.append(2 - alpha)
```

CPU times: user 28min 33s, sys: 42.9 s, total: 29min 16s

Wall time: 2h 2min 23s

```
[ ]: import pandas as pd
```

```
[ ]: for i, k in enumerate(ks):  
    df = f"escape_basin_i={i}.dat"  
    df = pd.read_csv(df, header=None, sep=r"\s+")  
    escape_basins[i] = np.array(df[2]).reshape(grid_size, grid_size)
```

```
[ ]: for i, k in enumerate(ks):  
    bm = BasinMetrics(escape_basins[i, :, :])  
    basin_entropy = bm.basin_entropy(5, log_base=2)  
    Sb.append(basin_entropy[0])  
    Sbb.append(basin_entropy[1])  
    eps, f = bm.uncertainty_fraction(x_grid, y_grid, )  
    alpha, _ = np.polyfit(np.log(eps), np.log(f), 1)  
    D.append(2 - alpha)
```

```
[ ]: array = escape_basins.reshape(len(ks), grid_size ** 2)  
prob_0 = np.sum(array == -1, axis=1) / (grid_size ** 2)  
prob_1 = np.sum(array == 0, axis=1) / (grid_size ** 2)  
prob_2 = np.sum(array == 1, axis=1) / (grid_size ** 2)
```

```
[71]: ps = PlotStyler(ticks_on_all_sides=False)  
ps.apply_style()  
  
# Create figure  
fig = plt.figure(figsize=(12, 4))  
  
gs = gridspec.GridSpec(2, 2)  
  
ax = []  
ax.append(fig.add_subplot(gs[:, 0]))  
ax.append(fig.add_subplot(gs[0, 1]))  
ax.append(fig.add_subplot(gs[1, 1]))  
ps.set_tick_padding(ax[0], pad_x=8)  
ps.set_tick_padding(ax[2], pad_x=8)  
  
width = ks[1] - ks[0]  
ax[0].bar(ks, prob_0, label='State 0', linewidth=1., edgecolor='black',  
    ↪width=width, align='edge', color="green")  
ax[0].bar(ks, prob_1, bottom=prob_0, label='State 1', linewidth=1.,  
    ↪edgecolor='black', width=width, align='edge', color="gold")  
ax[0].bar(ks, prob_2, bottom=prob_0 + prob_1, label='State 2', linewidth=1.,  
    ↪edgecolor='black', width=width, align='edge', color="blue")
```

```

ax[0].set_xlim(0, 1)
ax[0].set_xlabel(" $k$ ")
ax[0].set_ylim(0, 1)
ax[0].set_ylabel("Basin stability")

ax[1].plot(ks, Sb, "o-", color="blueviolet", label=" $S_b$ ")
ax[1].plot(ks, Sbb, "o-", color="maroon", label=" $S_{bb}$ ")
ax[1].set_ylabel("Basin entropy")
ax[1].legend(loc="lower right", frameon=False)
ax[1].set_xticklabels([])
ax[1].set_yticks([0, 0.5, 1, 1.5])

ax[2].plot(ks, D, "o-", color="blueviolet")
ax[2].set_ylabel(" $d$ ")
ax[2].set_xlabel(" $k$ ")
ax[2].set_yticks([1.4, 1.6, 1.8, 2])

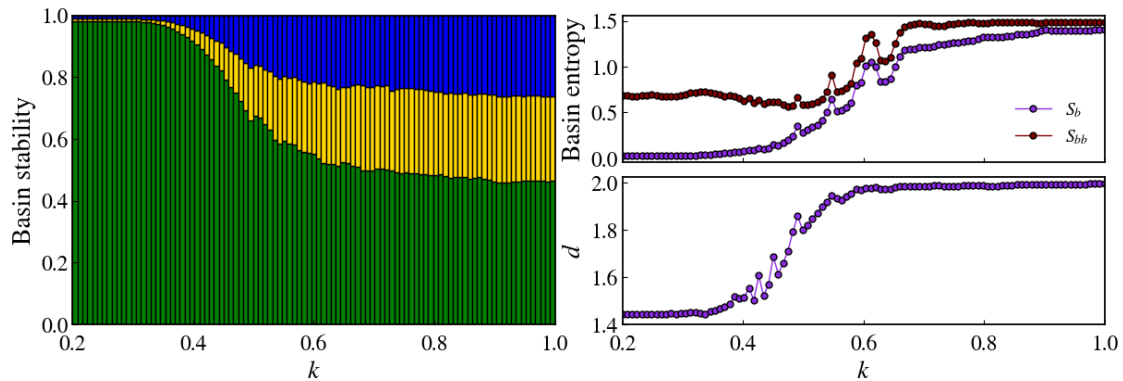
[ax[i].set_xlim(min(ks), max(ks)) for i in range(len(ax))]

plt.subplots_adjust(left=0.056, bottom=0.145, right=0.987, top=0.978, wspace=0.
↪14, hspace=0.1)

plt.savefig("fig14.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



5 Continuous-time systems

5.1 Lorenz system

```
[34]: cds.available_models()
```

```
[34]: ['lorenz system', 'henon heiles', 'rossler system', '4d rossler system']
```

```
[35]: cds.available_integrators()
```

```
[35]: ['rk4', 'rk45']
```

```
[37]: ds = cds(model="lorenz system")
```

```
[38]: time_steps = [0.05, 0.01, 0.005]
      tols = [[1e-6, 1e-3], [1e-8, 1e-5], [1e-10, 1e-7]]
      total_time = 80
      u = [0., 0.1, 0.]
      parameters = [10, 28, 8/3]
```

```
[41]: ds.integrator("rk4", time_step=0.01)
      trajectory = ds.trajectory(u, total_time, parameters=parameters)
      trajectory.shape
```

```
[41]: (8000, 4)
```

```
[42]: ds.integrator("rk45", atol=1e-8, rtol=1e-5)
      trajectory = ds.trajectory(u, total_time, parameters=parameters)
      trajectory.shape
```

```
[42]: (2388, 4)
```

```
[44]: %%time
      trajectories = []
      for i in range(3):
          ds.integrator("rk4", time_step=time_steps[i])
          trajectory = ds.trajectory(u, total_time, parameters=parameters)
          trajectories.append(trajectory)
          ds.integrator("rk45", atol=tols[i][0], rtol=tols[i][1])
          trajectory = ds.trajectory(u, total_time, parameters=parameters)
          trajectories.append(trajectory)
```

CPU times: user 60.8 ms, sys: 2.55 ms, total: 63.3 ms

Wall time: 77.7 ms

```
[82]: ps = PlotStyler(fontsize=18, linewidth=0.75, ticks_on_all_sides=False)
      ps.apply_style()

      fig, ax = plt.subplots(2, 3, sharex=True, sharey=True, figsize=(10, 5))
      for i in range(3):
          ds.integrator("rk4", time_step=time_steps[i])
          trajectory = ds.trajectory(u, total_time, parameters=parameters)
          ax[0, i].plot(trajectory[:, 1], trajectory[:, 3], "k")

          ds.integrator("rk45", atol=tols[i][0], rtol=tols[i][1])
          trajectory = ds.trajectory(u, total_time, parameters=parameters)
```

```

ax[1, i].plot(trajecory[:, 1], trajecory[:, 3], "k")

ax[1, i].set_xlabel("$x$")

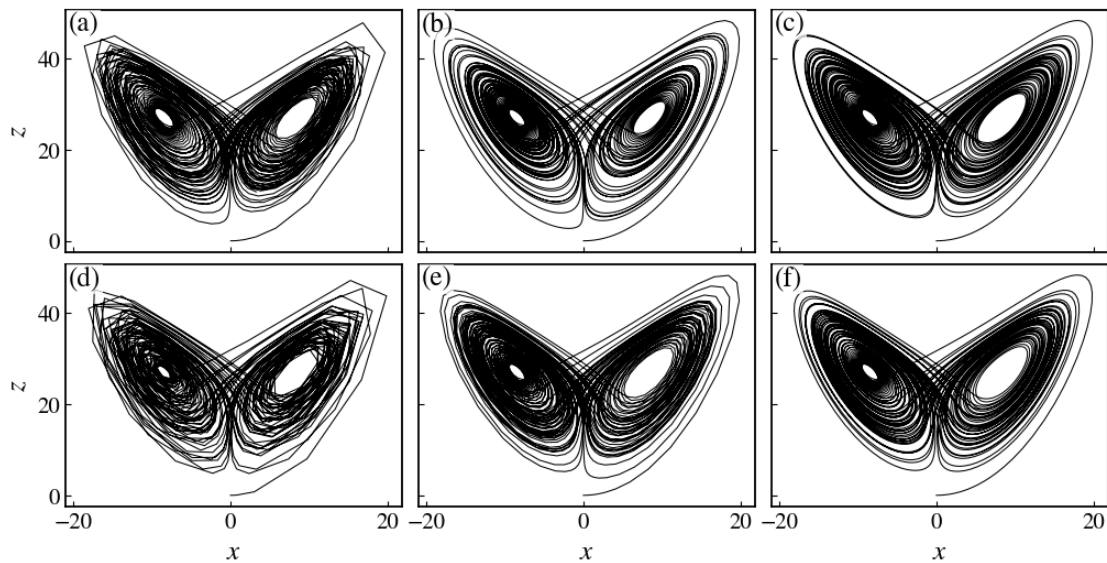
ax[0, 0].set_ylabel("$z$")
ax[1, 0].set_ylabel("$z$")
plt.subplots_adjust(left=0.05, bottom=0.095, right=0.9975, top=0.9975, hspace=0.
    ↪05, wspace=0.05)

xbox = 0.0085
ybox = 0.911
bbox = {"facecolor": "w", "alpha": 0.75, "linewidth": 0.0, "pad": 1}
for i in range(6):
    ax.flat[i].text(xbox, ybox, f"({ascii_lowercase[i]})", transform=ax.flat[i].
    ↪transAxes, bbox=bbox)

plt.savefig("fig15.png", dpi=400)

```

<Figure size 640x480 with 0 Axes>



5.2 Rössler system

```
[163]: from numba import njit
```

```
[164]: @njit
def rossler_system(time, u, parameters):

    a, b, c = parameters

```

```

dudt = np.zeros_like(u)

dudt[0] = -u[1] - u[2]
dudt[1] = u[0] + a * u[1]
dudt[2] = b + u[2] * (u[0] - c)

return dudt

@njit
def rossler_system_jacobian(time, u, parameters):

    a, b, c = parameters

    neq = len(u)
    J = np.zeros((neq, neq), dtype=np.float64)

    J[0, 0] = 0
    J[0, 1] = -1
    J[0, 2] = -1

    J[1, 0] = 1
    J[1, 1] = a
    J[1, 2] = 0

    J[2, 0] = u[2]
    J[2, 1] = 0
    J[2, 2] = u[0] - c

    return J

```

```

[165]: ds = cds(equations_of_motion=rossler_system, system_dimension=3,
↳number_of_parameters=3)
ds.integrator("rk4", time_step=0.005)

```

```

[166]: # Initial conditions
u = [0.1, 0.1, 0.1]

# Total and transient time
total_time = 2000
transient_time = 1000
parameters = [0.15, 0.20, 10.0]

```

```

[168]: %%time
trajectory = ds.trajectory(u, total_time, parameters=parameters,
↳transient_time=transient_time)

```

CPU times: user 235 ms, sys: 9.83 ms, total: 245 ms
Wall time: 245 ms

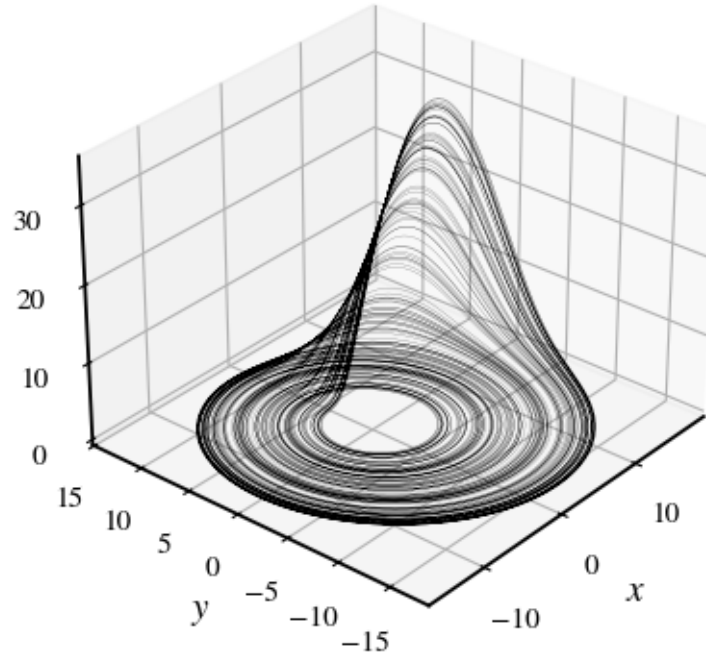
```
[14]: ps = PlotStyler(fontsize=14, linewidth=0.1)
      ps.apply_style()

      # Create the 3D figure and axis
      fig, ax = plt.subplots(subplot_kw={'projection': '3d'}, figsize=(5, 4))

      # Plot the trajectory
      ax.plot(trajectory[:, 1], trajectory[:, 2], trajectory[:, 3],
              label='Trajectory', color="k")

      # Set the labels and view angle
      ax.set_xlabel("$x$")
      ax.set_ylabel("$y$")
      ax.set_zlabel("$z$")
      ax.view_init(elev=30, azimuth=-140)
      plt.tight_layout()
      plt.savefig("fig16.png", dpi=400)
```

<Figure size 640x480 with 0 Axes>



5.3 Lyapunov exponents

```
[5]: from dynamicalsys import ContinuousDynamicalSystem as cds
total_time = 2000
transient_time = 1000
```

```
[6]: ds = cds(model="lorenz system")
u = [0., 0.1, 0.]
parameters = [16, 45.92, 4]
ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, log_base=2)
```

```
[6]: array([ 2.16623420e+00, -5.95950415e-04, -3.24609067e+01])
```

```
[7]: ds = cds(model="rossler system")
u = [0.1, 0.1, 0.1]
parameters = [0.15, 0.20, 10.0]
ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, log_base=2)
```

```
[7]: array([ 1.28949638e-01,  1.49355076e-03, -1.41369173e+01])
```

```
[8]: ds = cds(model="4d rossler system")
u = [-20., 0, 0., 15.]
parameters = [0.25, 3.0, 0.5, 0.05]
ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, log_base=2)
```

```
[8]: array([ 1.72511105e-01,  3.63638738e-02,  4.73195157e-03, -3.88752036e+01])
```

5.3.1 Rössler system

```
[54]: ds = cds(equations_of_motion=rossler_system, jacobian=rossler_system_jacobian,
↳system_dimension=3, number_of_parameters=3)
ds.integrator("rk4", time_step=0.01)
```

```
[55]: # Initial conditions
u = [0.1, 0.1, 0.1]

# Total and transient time
total_time = 2000
transient_time = 1000
parameters = [0.15, 0.20, 10.0]
```

```
[56]: ds.lyapunov(u, total_time, parameters, transient_time, log_base=2)
```

```
[56]: array([ 1.28949638e-01,  1.49355076e-03, -1.41369173e+01])
```

```
[57]: # Initial conditions
u = [0.1, 0.1, 0.1]

# Total and transient time
total_time = 20000
transient_time = 10000
parameters = [0.15, 0.20, 10.0]
```

```
[58]: %%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters, transient_time, log_base=2)
```

4.52 s ± 33.7 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)

5.3.2 Lorenz system

```
[69]: ds = cds(model="lorenz system")
ds.integrator("rk4", time_step=0.01)
```

```
[70]: # Initial conditions
u = [0., 0.1, 0.]

# Total and transient time
total_time = 2000
transient_time = 1000
parameters = [16, 45.92, 4]
```

```
[61]: ds.lyapunov(u, total_time, parameters, transient_time, log_base=2)
```

```
[61]: array([ 2.16623420e+00, -5.95950415e-04, -3.24609067e+01])
```

```
[62]: # Initial conditions
u = [0., 0.1, 0.]

# Total and transient time
total_time = 20000
transient_time = 10000
parameters = [16, 45.92, 4]
```

```
[63]: %%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters, transient_time, log_base=2)
```

4.59 s ± 148 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)

5.3.3 Rössler system 4D

```
[64]: ds = cds(model="4d rossler system")
ds.integrator("rk4", time_step=0.01)
```

```
[65]: # Initial conditions
u = [-20., 0, 0., 15.]

# Total and transient time
total_time = 2000
transient_time = 1000
parameters = [0.25, 3.0, 0.5, 0.05]

[66]: ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, log_base=2)

[66]: array([ 1.72511105e-01,  3.63638738e-02,  4.73195157e-03, -3.88752036e+01])
```

```
[ ]: # Initial conditions
u = [-20., 0, 0., 15.]

# Total and transient time
total_time = 20000
transient_time = 10000
parameters = [0.25, 3.0, 0.5, 0.05]

[ ]: %%timeit -n 5 -r 10
ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, log_base=2)
```

6.13 s ± 204 ms per loop (mean ± std. dev. of 10 runs, 5 loops each)

5.3.4 Average Lyapunov exponents

```
[28]: total_time = 20000
transient_time = 10000
n_runs = 50

[24]: ds_rs = cds(model="rossler system")
u0_rs = [0.1, 0.1, 0.1]
rs = [0.15, 0.20, 10.0]

ds_ls = cds(model="lorenz system")
u0_ls = [0.0, 0.1, 0.0]
ls = [16, 45.92, 4]

ds_rs4d = cds(model="4d rossler system")
u0_rs4d = [-20., 0, 0., 15.]
rs4d = [0.25, 3.0, 0.5, 0.05]

[50]: lyapunovs_rs = np.zeros((n_runs, len(u0_rs)))
lyapunovs_ls = np.zeros((n_runs, len(u0_ls)))
lyapunovs_rs4d = np.zeros((n_runs, len(u0_rs4d)))
```

```
[51]: %%time
      for i in range(n_runs):
          np.random.seed(i)
          u_rs = u0_rs + np.random.uniform(-1, 1, len(u0_rs)) * 1e-3
          u_ls = u0_ls + np.random.uniform(-1, 1, len(u0_ls)) * 1e-3
          u_rs4d = u0_rs4d + np.random.uniform(-1, 1, len(u0_rs4d)) * 1e-3
          lyapunovs_rs[i, :] = ds_rs.lyapunov(u_rs, total_time, parameters=rs,
          ↪transient_time=transient_time, log_base=2)
          lyapunovs_ls[i, :] = ds_ls.lyapunov(u_ls, total_time, parameters=ls,
          ↪transient_time=transient_time, log_base=2)
          lyapunovs_rs4d[i, :] = ds_rs4d.lyapunov(u_rs4d, total_time,
          ↪parameters=rs4d, transient_time=transient_time, log_base=2)
```

CPU times: user 13min 10s, sys: 3.28 s, total: 13min 13s
 Wall time: 13min 15s

```
[52]: for i in range(3):
      print(np.mean(lyapunovs_rs, axis=0)[i], np.std(lyapunovs_rs, axis=0)[i])
```

```
0.12814572986552566 0.0024411585163317353
0.00013826329742290378 0.00016190300381474764
-14.139155311918328 0.002640979740634374
```

```
[53]: for i in range(3):
      print(np.mean(lyapunovs_ls, axis=0)[i], np.std(lyapunovs_ls, axis=0)[i])
```

```
2.1678335808971445 0.00209511778657654
-0.0001573472147848526 0.0001576974567301843
-32.4629446284306 0.002088111268761096
```

```
[54]: for i in range(4):
      print(np.mean(lyapunovs_rs4d, axis=0)[i], np.std(lyapunovs_rs4d, axis=0)[i])
```

```
0.16141591603316185 0.003992117611179678
0.030927823306343596 0.0020594155886780037
-0.00016072498262674877 0.000277394149408847
-35.824094929850645 0.6160052794845143
```

5.4 SALI and LDI

```
[3]: from dynamicalsys import ContinuousDynamicalSystem as cds
      total_time = 20000
      transient_time = 10000
```

```
[23]: ds = cds(model="lorenz system")
      ds.integrator("rk45", atol=1e-10, rtol=1e-8)
```

```
[ ]: u = [0.0, 0.1, 0.0]
      parameters = [10, 33.3, 8/3]
```

```
[51]: %%time
trajectory = ds.trajectory(u, transient_time + 200, parameters=parameters,
↳transient_time=transient_time)
```

CPU times: user 2.41 s, sys: 9.32 ms, total: 2.42 s
Wall time: 2.42 s

```
[44]: %%time
lyapunov = ds.lyapunov(u, total_time, parameters=parameters,
↳transient_time=transient_time, return_history=True)
```

CPU times: user 4.27 s, sys: 33.6 ms, total: 4.3 s
Wall time: 4.31 s

```
[160]: %%time
sali = ds.SALI(u, total_time, parameters=parameters,
↳transient_time=transient_time, return_history=True)
```

CPU times: user 2.31 s, sys: 15.9 ms, total: 2.33 s
Wall time: 2.33 s

```
[161]: %%time
ldi_2 = ds.LDI(u, total_time, 2, parameters=parameters,
↳transient_time=transient_time, return_history=True)
```

CPU times: user 2.37 s, sys: 15.2 ms, total: 2.38 s
Wall time: 2.39 s

```
[162]: %%time
ldi_3 = ds.LDI(u, total_time, 3, parameters=parameters,
↳transient_time=transient_time, return_history=True)
```

CPU times: user 2.34 s, sys: 10.9 ms, total: 2.35 s
Wall time: 2.35 s

```
[59]: sali.shape
```

```
[59]: (608, 2)
```

```
[117]: ldi_3[-1, 0]
```

```
[117]: 10002.457081772074
```

```
[159]: ps = PlotStyler(linewidth=1, fontsize=17, ticks_on_all_sides=False)
ps.apply_style()

fig, ax = plt.subplots(1, 3, figsize=(10, 4))
[ps.set_tick_padding(ax[i], pad_x=7) for i in range(len(ax))]
ax[0].plot(trajectory[:, 1], trajectory[:, 3], "k", lw=0.25)
ax[0].set_xlabel("$x$")
```

```

ax[0].set_ylabel("$z$")

ax[1].plot(lyapunov[:, 0], lyapunov[:, 1], c="darkgreen", label=r"$\lambda_1$")
ax[1].plot(lyapunov[:, 0], lyapunov[:, 2], c="gold", label=r"$\lambda_2$")
ax[1].plot(lyapunov[:, 0], 0.05 * lyapunov[:, 3], c="blue", label=r"$0.05\lambda_3$")
ax[1].set_ylim(-1, 1.1)
ax[1].legend(loc="upper right", ncol=1, frameon=False)
ax[1].set_xlim(transient_time, total_time)
ax[1].set_xlabel("$t$")
ax[1].set_ylabel("Lyapunov exponents")

ax[2].plot(sali[:, 0], sali[:, 1], "k", label="SALI")
ax[2].plot(ldi_2[:, 0], ldi_2[:, 1], "r", label="LDI$_2$")
x_new = np.linspace(0, 1000, 1000)
y_new = np.exp(-(lyapunov[-1, 1]) * x_new)
ax[2].plot(x_new + transient_time, y_new, "k--", label=r"$e^{-\lambda_1 t}$")

ax[2].set_yscale("log")
ax[2].set_ylim(1e-16, 1e1)
ax[2].set_xlim(transient_time, sali[-1, 0])
ax[2].set_xlabel("$t$")
ax[2].set_ylabel("SALI and LDI")

ax[2].plot(0, 0, "b-", label="LDI$_3$")
ax[2].plot(0, 0, "b--", label=r"$e^{-(2\lambda_1 - \lambda_3)t}$")
ax[2].legend(loc="lower left", frameon=False)

# ax_inset = ax[2].inset_axes([0.2, 0.1, 0.3, 0.3])
ax_inset = ax[2].inset_axes([0.5, 0.7, 0.48, 0.28])
ax_inset.plot(ldi_3[:, 0], ldi_3[:, 1], "b")
# ax_inset.set_xticks([transient_time, 10002], [1, 2])
ax_inset.set_yscale("log")
ax_inset.set_xlim(transient_time, ldi_3[-1, 0])
ax_inset.set_ylim(1e-16, 1e1)
ax_inset.set_yticks([1e-16, 1e-8, 1e-0])
x_new = np.linspace(0, 1000, 1000)
y_new = np.exp(-(2 * lyapunov[-1, 1] - lyapunov[-1, 3]) * x_new)
ax_inset.plot(x_new + transient_time, y_new, "b--", label=r"$e^{-(\lambda_1 t)}$")

bbox = {"facecolor": "w", "linewidth": 0, "alpha": 0.75, "pad": 1}
xbox = 0.0099
ybox = 0.9457

for i in range(len(ax)):
    ax[i].text(xbox, ybox, f"({ascii_lowercase[i]})", transform=ax[i].
        ↪transAxes, bbox=bbox)

```

```
plt.subplots_adjust(left=0.0475, bottom=0.125, right=0.998, top=0.98, wspace=0.
↪3)
```

```
plt.savefig("fig17.png", dpi=500)
```

<Figure size 640x480 with 0 Axes>

